

AstroBurst

High-Performance Astronomical Image Processor

Technical Document — Release v0.6.7.0

Samuel Krieger Bonini

October 2026

Field	Value
Document Version	0.6.7.0
Release	v0.6.7.0 (Rust crate and Tauri bundle 0.6.7)
Date	10 October 2026
Author	Samuel Krieger Bonini
Stack	Rust (stable) · Tauri v2.10 · React 19 · TypeScript 5.7
Platforms	Windows · macOS · Linux
License	AGPL-3.0-or-later
Repository	https://github.com/samuelkriegerbonini-dev/AstroBurst

Contents

1	Executive Summary	5
1.1	What's New since v0.6.0	5
1.2	What's New in v0.6.7	6
1.3	v0.6.0 in Brief	8
1.4	What's New in v0.5.8	9
1.5	What's New in v0.5.5	10
1.6	What's New in v0.4.6	10
1.7	What's New in v0.4.5	11
1.8	What's New in v0.4.2	11
1.9	What's New in v0.4.0	11
2	System Architecture	12
2.1	Backend (Rust)	12
2.2	Non-Destructive Composite Pipeline (v0.4.5)	13
2.3	Frontend (React + TypeScript)	13

2.4	Tool Windows and the Dock (v0.6.5 to v0.6.7)	14
2.5	IPC Strategy	15
3	Mathematical Foundations	15
3.1	Screen Transfer Function (STF)	15
3.2	Asinh Stretch	16
3.3	Masked Stretch (v0.4.2)	16
3.4	Cross-Correlation Alignment (Phase Correlation)	16
3.5	Star-Based Affine Alignment (v0.4.0)	17
3.6	Sigma-Clipped Stacking	17
3.7	Channel Blending (v0.4.2)	17
3.8	White Balance (v0.4.0, non-destructive v0.4.5)	17
3.9	SCNR (v0.4.0, non-destructive v0.4.5)	18
3.10	Drizzle Integration	18
3.11	Richardson-Lucy Deconvolution	18
3.12	Background Extraction	18
3.13	Wavelet Denoising	18
3.14	Calibration Pipeline (revised in v0.6.7)	19
3.15	Star Detection	19
3.16	SPCC: Blackbody Colour Calibration (v0.4.2, revised in v0.6.7)	19
3.17	Pixel Rejection Families (v0.6.0)	20
3.18	Frame Normalization (v0.6.0)	22
3.19	Thin-Plate Spline Background Extraction — DBE (v0.6.0)	23
3.20	Aperture Photometry with Fractional Pixel Weights (v0.6.0)	24
3.21	Barycentric and Heliocentric Radial Velocity (v0.6.0)	25
3.22	Moment Maps (v0.6.0)	27
3.23	Wavelet Per-Scale Detail Bias (v0.6.0)	28
3.24	Local Histogram Equalization and HDR Multiscale Transform (v0.6.0)	29
3.25	Effective Gain for Photometry (v0.6.7)	30
3.26	Level Matching and the Composite STF Link (v0.6.7)	31
3.27	Gaussian Line-Fit Maps (v0.6.6)	32
3.28	Tabulated Spectral Axes (v0.6.7)	33
3.29	Reprojection Through the WCS (v0.6.7)	33
3.30	Exact Rotations and Flips (v0.6.7)	34
3.31	gWCS Evaluation and the Exact Inverse (v0.6.7)	35
3.32	TAN-SIP Fit of a gWCS (v0.6.7)	36

4	World Coordinate Systems	36
4.1	Header WCS: the Celestial Axis Pair (v0.6.7)	36
4.2	SIP Fit Residual Cards (v0.6.7)	37
4.3	gWCS Sources and Policy (v0.6.7)	37
4.4	Derived FITS Outputs	38
4.5	Plate Solving and Write WCS	39
5	Measurement and Science Analysis	39
5.1	The Measurement Contract	39
5.2	Regions and Exclusions	40
5.3	Photometry	40
5.4	Catalogs and the Disk Cache (v0.6.7)	40
5.5	Filters	40
5.6	Spectra	41
5.7	Cubes	41
5.8	Ramps (v0.6.6)	41
5.9	Time Series, Observation Geometry and the Log	42
6	Calibration, Stacking and Compose	42
6.1	Frame Cards and Consistency Checks (v0.6.7)	42
6.2	Stack Outputs and Combination (v0.6.7)	43
6.3	Batch Pipeline	44
6.4	Compose Wizard	44
7	Processing Tools	45
7.1	Curves and Levels on a File	45
7.2	Background and De-Banding on a File	45
7.3	Geometry: Rotate and Flip	45
8	Command Module Reference	46
9	FITS and ASDF I/O	46
9.1	Image References (v0.6.0)	46
9.2	Lossless Integer Planes (v0.6.0)	46
9.3	Multi-Extension Output (v0.6.0)	48
9.4	Binary Tables (v0.6.5, v0.6.7)	48
9.5	The JWST ASDF Extension (v0.6.7)	48
9.6	Ramps (v0.6.6)	48

10 WebGPU Compute Pipeline	48
10.1 Display Path Unification (v0.6.0, extended in v0.6.3 and v0.6.5)	48
11 Headless HTTP Server	49
11.1 Session Model	50
11.2 The v2 API Surface	51
11.3 Job Model	52
11.4 What Is Shared, and What Is Not	53
11.5 Error Model	53
12 Testing and Continuous Integration	54
13 Performance Benchmarks	55
14 Dependencies	56
15 Known Limitations	56
16 Roadmap	59
A Sample Test Data	60

1 Executive Summary

AstroBurst is a cross-platform desktop application for viewing, measuring, processing and composing astronomical FITS and ASDF images. The processing runs in Rust, the interface is drawn by React, and Tauri packages both as a desktop application. A WebGPU shader applies the display stretch and colormap; that is the only use of the GPU, and every algorithm also runs without it.

The application targets two user segments:

- Professional astronomers working with observatory data (JWST, Hubble, Roman Space Telescope, ground-based IFU instruments)
- Advanced astrophotographers processing deep-sky narrowband and broadband imaging data

Release 0.6.7.0 exposes **140 backend commands** through a typed TypeScript service layer of 32 modules, a ten-step compose wizard and a processing tab whose steps form a chain. The same core is served over HTTP by an optional headless server (Section 11), so region statistics, rendering, histograms, cutouts, WCS queries and batch stacking are reachable from a script without a display. At release the library suite runs 2389 tests (plus 29 real-data and timing tests marked `#[ignore]`), the server binary adds 190 and the frontend suite runs 2962 cases in 199 files (Section 12).

The releases after 0.6.0 moved the project in three directions. Measurement: aperture and batch photometry with a header-derived gain model, calibrated region fluxes, line measurement and per-spaxel line-fit maps, position-velocity diagrams and time-series photometry, all logged. Instrument-level data: JWST `_uncal` ramps with a quick-look slope, pipeline `x1d` tables, tabulated spectral axes, and the JWST and Roman generalized WCS (gWCS) evaluated from its own transform chain instead of an approximation. Integrity of the astrophotography path: calibration frames are checked against each other before any master is built, channels are aligned through their WCS, and frames that cannot be registered are left out of a stack and reported, never stacked silently.

1.1 What's New since v0.6.0

One line per release; the full entries are in `CHANGELOG.md`. Version numbers with four components are point releases of the 0.6 line.

- **0.6.1** — The CPU and GPU previews show the same image after every processing step.
- **0.6.2 (2026-09-25), measurement round** — An interactive profile plot (hover, zoom, pan, CSV and PNG export); aperture photometry controls with a curve of growth and EE50/EE80 radii, and a Photometry table for batch measurement; calibrated region statistics (Jy, AB magnitude, mag/arcsec², area, centre RA/Dec, position angle); sky geometry (`world_to_pixel_cmd`, separation and position angle, WCS orientation, a Targets panel, a compass and scale bar); spectral line measurement with a bounded Gauss-Newton fit (`math::gauss_newton`); contours; elliptical surface-brightness profiles; time-series photometry; a pixel table; spectrum-linked channel navigation.
- **0.6.3 (2026-09-25), science round 2** — A rest-frame line list; spectrum export and comparison; observation geometry (UTC/TT/TDB, BJD_TDB, HJD_UTC, local sidereal time, airmass, parallactic angle, Sun and Moon); position-velocity diagrams; four diverging colormaps and symmetric limits (13 colormaps in total); the measurement log; and an interface round (status strip, press-and-hold Original, Compare wipe, 0-based coordinate labels, modal command palette).

- **0.6.4 (2026-09-26)** — The Processing tab works on the colour composite built in Compose, with its own processing chain and a revert to the original; Masked Stretch protection rewritten.
- **0.6.5.0 (2026-10-02)** — Dockable tool windows; an Align overlay and a Crop editor in the wizard; dependency warnings between wizard steps; the JWST pipeline `x1d` in the spectrum comparison; a wavelength readout from a `WAVELENGTH` extension; exclude regions; a colorbar; a first Match levels (replaced in 0.6.7); Write WCS after a plate solve; the Pipeline writes its masters; one-shot-colour lights refused before alignment and drizzle; display-referred STF export headers.
- **0.6.6 (2026-10-03)** — Per-spaxel line-fit kinematic maps for IFU cubes with a spaxel inspect plot; 27 lines in the line list; JWST `_uncal` and Roman L1 ramps with a quick-look slope compared against the official `_rate`; synthetic stacks that vary from frame to frame and list each frame in a manifest.
- **0.6.6.3 (2026-10-04)** — Analysis becomes five dock tools (Image, Astrometry, Photometry, Cube, Log); PixelMath `$T` always names the input image; the `PUPIL` filter rule for NIRCам pupil-wheel filters; decimal-comma validation; the Masked Stretch black point follows the Auto STF shadow rule.
- **0.6.7.0 (2026-10-10)** — Detailed in Section 1.2.

1.2 What's New in v0.6.7

Version 0.6.7.0 is a correctness and integrity round. Each item below names what was wrong or missing before; the technical sections give the rules.

- **Astrometry and WCS** (Sections 4, 3.31, 3.32)
 - **Celestial axis pair required** — A header without `CTYPE1/CTYPE2`, with a non-celestial second axis, with latitude on axis 1 or with two different projection codes is refused with a reason. Before, a missing `CTYPE1` was read as `RA---TAN` and `CTYPE2` was never read, so a 2-D spectrum without `CTYPE` received invented sky coordinates.
 - **SIP fit residual** — `SIPMXERR/SIPIVERR` are read, reported by `get_wcs_info` and the server `wcs` route, shown in the Info orientation line, grouped under “WCS / Astrometry” in the Header Explorer, rescaled by drizzle and resample, and treated as WCS cards when a WCS is replaced.
 - **gWCS evaluator** — The JWST `_cal` gWCS (read from the `ASDF` extension of the FITS file) and the Roman L2 `roman.meta.wcs` are evaluated exactly: polynomial distortion, mappings, 3-D rotation sequences and the TAN projection, with an exact inverse by Newton iteration on the sphere. Roman `_cal.asdf` files, which previously opened without any WCS, now have sky coordinates, a grid and catalog overlays; JWST `_cal` files use the gWCS instead of the header TAN-SIP and report the difference between the two.
 - **TAN-SIP fit for derived FITS** — A Roman image gets a TAN-SIP header fitted to its gWCS (degree chosen automatically to 0.01 px), so cutouts, crops and exports carry a valid header WCS; a JWST cutout receives the pipeline’s own residual cards.
 - **Arithmetic gWCS operators** — The astropy compound operators (`add`, `subtract`, `multiply`, `divide`, `power`) and `Const1D/Const2D` are evaluated, so tweakreg-corrected `_crf` products use their gWCS instead of falling back to the header SIP.
 - **Plate solving** — Solve and Write WCS run on the image being measured (a stack or a processed result), not on the original file; every GET to nova.astrometry.net is retried on transient failures (Section 4.5).
- **Measurement** (Section 5)
 - **Measurements follow the image on screen** — The Info panel sky readout reads

- the analysis target and hides itself for a target without a celestial WCS (a PV diagram); Statistics, region statistics, profiles and the line sky readout re-run when a processing step rewrites the same file; a click on a result whose grid differs from the cube (a PV diagram) no longer extracts the spectrum of an unrelated spaxel.
- **Gain model for photometry** — The gain field is prefilled from the header (`EGAIN`, `ATODGAIN`, `CCDGAIN`, `ADCGAIN`, never the camera `GAIN` setting), scaled for AstroBurst stacks by `NCOMBINE` and the combine method, and replaced by the `ERR` plane when there is one. Click, batch and time-series photometry warn when the errors leave out source photon noise (Section 3.25).
 - **Filters in Rust** — A Rust mirror of the filter-wavelength table (74 codes, NIRISS F158M and F380M and 14 HST broadband codes added), a parity test against the TypeScript table, and the `PUPIL` rule in the backend, so a time series no longer mixes an F444W+F470N frame with an F444W frame.
 - **Catalog disk cache** — Gaia DR3 cones are cached on disk (64 MiB, oldest first) and served memory → disk → network; the Vizier endpoint and the cache directory can be overridden; click photometry matches against one field cone instead of one request per star; SPCC uses the same cache.
 - **Tables and -TAB axes** — Fixed-repeat array columns of binary tables are read (with `TDIM`); a `WAVE-TAB` spectral axis is evaluated from its `WCS-TABLE` per FITS WCS Paper III (Section 3.28).
 - **Calibration and stacking** (Section 6)
 - **Frame-card checks** — Binning, `GAIN`, `OFFSET`, dark exposures, flat filters and `IMAGETYP` are checked before any master is built, in the Calibration tab, the Pipeline and the server; mismatches that corrupt the result are refused, the others warned.
 - **Dark scale from exposures** — The Calibration tab no longer asks for a dark ratio; the scale is the exposure ratio when a bias is present and 1 otherwise. Exposure is read from the merged primary and SCI header, so a JWST `EXPOSURE=1` (a count) is no longer read as one second.
 - **Flat-darks and temperature groups** — Flat-darks are a fourth master; darks are grouped by sensor temperature and each light takes the nearest group.
 - **Provenance cards** — Stack, Pipeline and drizzle outputs write `NCOMBINE`, `EXPTIME`, `TOTEXP`, `ABCOMB`, `ABREJECT`, `ABNORM` and `ABDRZSCL`.
 - **Combine correctness** — Noise weights are divided by the square of the normalization scale; frames of different sizes are refused without alignment; the cosmetic zero-MAD fallback uses the 90th percentile; the Masked Stretch shadow guard no longer depends on the data range; cosmetic correction detects Bayer data from the header.
 - **Unregistered frames excluded** — The batch Pipeline leaves out every frame it could not register and reports each one by its position in the channel; an unknown alignment method name sent to Stack, Compose Align or the aligned-channel export is an error instead of a silent phase correlation.
 - **Compose** (Section 6.4)
 - **Match levels by median and MAD** — Each channel is scaled to the largest MAD and shifted to the reference median before blending; the composite FITS has balanced planes (Section 3.26).
 - **STF link rule** — The preview STF is unlinked when the composite planes' medians differ by more than three sigma or by a ratio above 1.5.
 - **SPCC at each channel's wavelength** — The blackbody model is evaluated at the effective wavelength of each input filter instead of fixed 640/530/460 nm, and the option is labelled “SPCC (blackbody approximation)”.
 - **Align by WCS** — Channels with a celestial WCS are reprojected onto the reference

- grid through the WCS, the residual is measured on point sources, and the reference is the finest pixel scale unless the user picks one (Section 3.29).
- **Run tokens, crop and export** — Wizard cache keys carry a per-run token so a late Align cannot overwrite a newer one; a stack that finishes after a Reset is discarded; Crop works on a single channel; star removal no longer marks Stretch done; Export shows and records the fixed channel mapping.
 - **Processing** (Section 7)
 - **Curves and Levels on a file** — A Curves panel applies levels and a tone curve to a mono or RGB FITS as a chain step.
 - **De-banding on a file** — The Background panel offers the de-band modes for a single frame and refuses dispersed exposures.
 - **Rotate and flip** — Rotations by 90, 180 and 270 degrees and flips with an exact WCS update, including the SIP polynomials (Section 3.30).
 - **Interface** — A first-run coachmark explains how to dock tool windows vertically and horizontally (Section 2.4).
 - **Headless server** — The same WCS loader (gWCS included), the frame-card checks and flat-darks in `pipeline/run`, the new WCS keys on the `wcs` route, and the catalog cache initialised at start (Section 11).

1.3 v0.6.0 in Brief

Version 0.6.0 was published as a preview of the 0.6 line; the Windows MSI bundle reported 0.6.0 because MSI versions cannot carry a pre-release identifier. Its content, condensed:

- **Viewer maturity** — Six scale modes (`mtf`, `linear`, `log`, `sqrt`, `asinh`, `power`), four limit modes (`minmax`, `zscale`, `percentile`, `user`) and nine colormaps defined once in `core/imaging/scale.rs` and shared by the GPU shader, the worker fallback and the server; pixel readout with units; cursor coordinates in ICRS, FK5, galactic or ecliptic.
- **Planes and DQ** — Image references (`<path>#hdu=<n>`, `<path>#array=<key>`), lossless integer planes, per-telescope DQ flag tables, DQ and ERR companions in the pixel readout, a DQ overlay and DQ-masked statistics.
- **Regions** — Seven shapes with handles, DS9 `.reg` import and export, per-region statistics with background annuli, radial profiles and line cuts, one geometry module shared with the server.
- **Processing parity** — The pixel rejection family and frame normalization (Sections 3.17, 3.18), masters integrated with the same kernel, cosmetic correction, DBE (Section 3.19), PixelMath, LHE and HDRMT (Section 3.24), the multiscale API, exact statistics with k -sigma noise.
- **Science analysis** — Photometric calibration from header zero points, fractional-pixel apertures (Section 3.20), the FITS spectral axis with velocity conventions and barycentric correction (Section 3.21), region spectra, range collapse and moment maps (Section 3.22), science cutouts, the WCS coordinate grid and the Gaia DR3 catalog panel.
- **WCS engine** — The `wcs` crate (`wcs-rs` and `mapproj`, about 20 projections, the CD, PC and CDELT conventions); SIP is applied by AstroBurst's own code because `mapproj` 0.4.0's SIP evaluator advances polynomial powers by repeated squaring.
- **GPU viewer parity** — The processed FITS becomes the raw-pixel source of the WebGPU renderer for every mono step.

1.4 What's New in v0.5.8

Version 0.5.8 is an audit release: the math, GPU flow, image-processing and performance paths were reviewed across 21 module groups with every finding reproduced by hand before being fixed, and the ASDF reader was validated against the ASDF Standard 1.5/1.6.

- **Drizzle kernel** — The Lanczos3 kernel was evaluated in output-pixel units, so at scale 3 (and any integer scale with zero offsets) 8/9 of the output pixels received zero weight and rendered as holes; the kernel argument and support now scale with the output grid
- **Sigma-clip floor** — Sigma-clip combine and the batch pipeline used a $1e-10$ sigma floor when the MAD is 0 (more than half the samples tied), rejecting every value that differed from the median; they now fall back to the mean absolute deviation and skip rejection when the scale is truly zero
- **Star removal** — The soft star mask was applied twice, since push-pull fill already blends by the mask, leaving $m - m^2$ of the star flux at every mask edge
- **GHS stretch** — $D = 0$ returned the raw, un-normalized data instead of the normalized identity, producing a white preview and export once the composite was cached
- **Wavelet reconstruction** — Negative pixels were clamped to 0 and NaN was rewritten as 0
- **Phase correlation** — A confident coarse shift was discarded whenever the fixed centre crop was featureless, falling back to identity or noise; the coarse estimate is now kept when the refine pass has no signal
- **Star detection** — Saturated flat-top stars were deblended into several detections; equal-valued adjacent maxima now collapse to a single seed
- **GPU/CPU preview parity** — IPC preview extrema were computed over all finite pixels while auto-STF came from padding-aware statistics, so the GPU render and the canonical PNG disagreed; NaN pixels were rewritten to 0.0 before upload and rendered gray; and linked STF on the GPU normalized each channel by its own range while the Rust path uses the combined range
- **WebGPU recovery** — A failed initialisation (no adapter, device creation error) was cached for the whole session; the retry path re-probes
- **Compressed FITS tiles** — Quantized GZIP_1/GZIP_2 tiles were decoded as raw f32 bit patterns (the integer width is now inferred from the tile byte count), ZBITPIX ± 64 was rejected for every codec including AstroBurst's own GZIP_2 files, the Rice decoder indexed the stream unchecked and panicked on short or corrupt tiles, and constant tiles quantized with scale 1 under SUBTRACTIVE_DITHER_1 gained ± 0.5 noise on decode
- **ASDF block handling** — Blocks are memory-mapped and decompressed lazily, so only the block behind the selected array is decoded (Roman L2 files carry six or more full-frame arrays), and uncompressed contiguous arrays are borrowed instead of copied three times
- **ASDF conformance** — #ASDF BLOCK INDEX trailers, CRLF line endings, tree padding, STREAMED blocks and `shape: ['*', ...]` are handled per spec; negative strides were silently read as contiguous; external block references now fail explicitly; gWCS chains under `roman.meta.wcs` are resolved with the Shift offset converted from 0-based gwcs pixels to 1-based CRPIX; YAML-tagged meta subtrees are flattened into the header instead of dropped; and allocation sizes from untrusted block headers are bounded
- **Headless server correctness** — Region, histogram, cutout and pixel endpoints validated inputs only after arithmetic that could wrap or allocate unbounded memory; statistics and histograms silently discarded every pixel $\leq 1e-7$, giving wrong results for bias-subtracted or difference images; binning by a non-divisible factor averaged overlapping windows; viewport auto-STF from crop statistics was applied with global normalization; and re-opening an image under an existing name returned the stale cached array
- **Job state** — Cancellation is observed by workers before results are published and terminal

transitions are atomic, so a late worker can no longer overwrite `cancelled`

- **Memory and throughput** — The batch stacking pipeline loads and stacks one channel at a time and returns 2048px previews instead of full-resolution base64 masters; drizzle scatters contributions directly into per-band accumulators in parallel instead of materialising every contribution as a tuple; and cube global statistics sample at most 32 frames with a pixel stride bounded to roughly 8M samples

1.5 What's New in v0.5.5

- **WebGPU RGB preview** — GPU-only `GpuRgbRenderer` renders full RGB composites from three `r32float` textures with per-channel STF in the WGSL shader (MTF identical to the mono shader, the CPU worker and the Rust backend); falls back to the PNG composite when WebGPU is unavailable, no adapter is found or the device is lost
- **Live per-channel RGB STF** — `RgbStfPanel` in the Analysis tab restretches the GPU composite live with no backend round-trip; the mono STF slider skips its backend render while a GPU preview is active
- **Linear RGB preview command** — `get_raw_rgb_pixels_preview(path?, max_dim)` returns a linear 3-channel float preview with per-channel min/max from a native RGB FITS or the composite cache (planar R|G|B behind a 32-byte header), with `encode_channel_preview` factored out of the mono encoder byte-identically
- **Canonical filter table** — `FILTER_WAVELENGTHS_NM` and `filter_to_wavelength_nm` in `types/constants.rs`, mirrored by `utils/filterWavelengths.ts`, normalize CLEAR, separators and dual filters, replacing two duplicated JWST tables
- **Dynamic channel Auto-Map** — Each non-narrowband, non-RGB filter gets its own wavelength-labelled channel grouped by nm, so NIRCam F090W–F444W yields five distinct channels
- **Auto (λ) blend** — Spreads any number of channels across R/G/B by wavelength, applied once for untouched broadband stacks and never overriding a palette or manual weights
- **Preview downsampling** — Nearest-neighbour replaced by a contrast-weighted hybrid (mean to max) area kernel that preserves faint point sources while anti-aliasing the background
- **STF GPU/CPU parity** — The CPU fallback worker no longer hard-zeros raw $\leq 1e-7$, which blacked out background-subtracted pixels that the GPU shader maps to gray; epsilons are aligned at $1e-8$
- **Preview normalization** — `data_min/data_max` describe the full image through a shared `scan_extrema` instead of the decimated subset, so the live preview matches the canonical PNG
- **GPU failure handling** — `GpuSingleton` exposes `onGpuLost` and an uncaptured-error handler so a dead device's resources are dropped and the CPU worker takes over, an oversized image degrades to the worker instead of a blank canvas, and switching files no longer leaves stale pixels blocking the fetch
- **Dead code removed** — The orphaned ZNCC WebGPU shader (`zncc_align.wgsl`), the unreachable client-side downsample worker and unused imports

1.6 What's New in v0.4.6

- **Tone curves (AdjustStep)** — Spline-based curve editor with per-channel R/G/B and linked RGB modes; non-destructive (reads `COMPOSITE_KEY`, applies STF + curves for preview only)
- **Auto-STF preview** — All linear-domain commands (blend, calibrate, WB, SCNR, reset) render preview with linked auto-STF, eliminating washed-out previews

- **Cache-only intermediate processing** — align, stack, calibrate, and background extraction store results in GLOBAL_IMAGE_CACHE; only PNG previews written to disk until explicit export
- **Narrowband detection via blend preset** — `isNarrowbandWorkflow` checks preset (SHO, HOO, Foraxx) as fallback when bin IDs are r/g/b; SCNR warning badge for narrowband workflows
- **Auto-STF propagation** — blend, calibrate, WB, and reset commands return `auto_stf` in response; StretchStep sliders initialize from auto-STF; ColorBalanceStep propagates post-WB STF to CompositeContext
- **Export pipeline fixes** — STF cache hit corruption fixed (linear data rendered directly when `applyStfStretch = false`); WCS header preservation for composite FITS; sentinel path handling
- **Star detection safety** — NaN guard (`is_finite && > 1e-7`), degenerate image early return, NaN-safe sort comparator
- **TypeScript strict mode** — 39 errors resolved (0 remaining); 22 missing lucide-react declarations added
- **Windows asset protocol** — Path safety with canonicalize, URL decode compatibility, async runtime
- **Flatpak/Flathub** — Submission pending approval for Linux distribution

1.7 What's New in v0.4.5

- **Non-destructive composite pipeline** — Immutable ORIG cache with idempotent WB and SCNR; `reset_wb_cmd` for one-click undo
- **STF rendering consistency** — GPU shader, CPU worker, and Rust backend produce pixel-identical output via symmetric denominator protection and unified padding threshold
- **Blend preset positional fallback** — SHO/HOO/Foraxx presets work with any bin configuration via positional weight mapping
- **Spectroscopy wavelength unit conversion** — Automatic CUNIT3 detection (M, NM, ANGSTROM, HZ) with display conversion
- **Export pipeline fix** — Exports use user STF params instead of identity, matching preview

1.8 What's New in v0.4.2

- **ComposeWizard** — 11-step pipeline (Channels, Stack, Align, Crop, BG, Blend, Color, Mask, Stretch, Adjust, Export)
- **Synthetic FITS generation** — Star field, PSF, and noise injection for testing
- **Narrowband palette system** — SHO, HOO, HOS, NaturalColor, Custom with auto-detection
- **Live composite STF re-stretch** — Per-channel shadow/midtone/highlight without re-composing
- **SPCC** — Spectrophotometric color calibration with optional Gaia DR3 TAP (vizier feature)
- **Masked stretch** — Iterative MTF with star mask protection
- **PSF enhancement** — Moment-based FWHM with subpixel peak estimation

1.9 What's New in v0.4.0

- **Star-based affine alignment** — Triangle asterism matching with RANSAC, automatic fallback to FFT phase correlation

- **Stability-based auto white balance** — Lowest MAD/median channel as reference
- **SCNR luminance redistribution** — ITU-R BT.709 weighted redistribution to R and B
- **Numerical correctness audit** — NaN ordering, Hann window, polynomial basis, RL regularization, phase correlation confidence

2 System Architecture

AstroBurst follows a strict backend/frontend separation, with all computation done in Rust and all rendering handled by the browser engine embedded in Tauri. The same Rust library has two front ends: the Tauri command layer for the desktop app and an HTTP server for scripts.

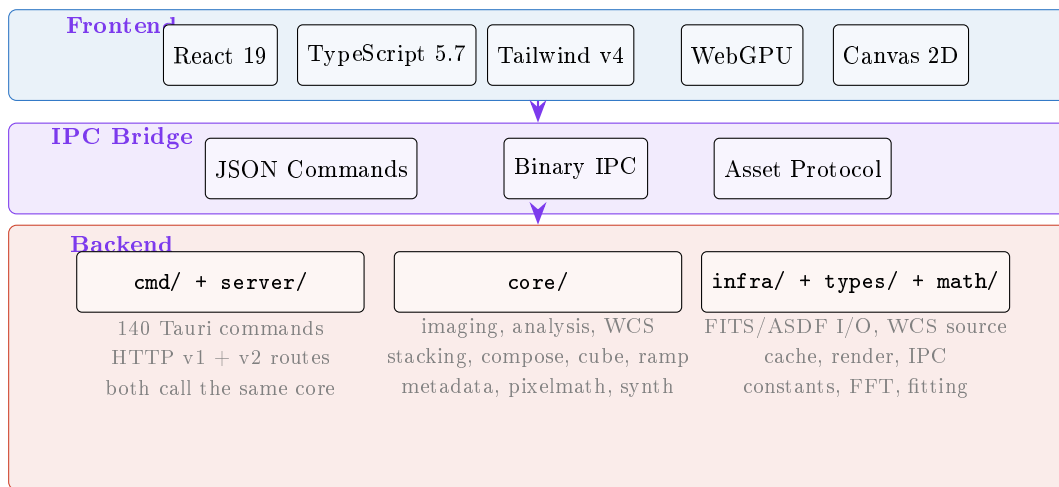


Figure 1: AstroBurst three-layer architecture

2.1 Backend (Rust)

The library crate (`astroburst_lib`) has five layers; two front ends sit on top of it.

- cmd/** Tauri command handlers grouped by domain, built only with the `tauri` feature (140 registered commands, Section 8). Each handler receives deserialized arguments, moves the work off the UI thread, delegates to `core/` and returns JSON or a binary buffer.
- server/** The headless HTTP front end, built only with the `server` feature (Section 11).
- core/** Computation, in eleven domains: **alignment** (phase correlation, triangle-asterism affine, WCS reprojection with the point-source residual), **analysis** (histogram, FFT, star detection, aperture photometry, the gain model, line measurement, x1d reader, subframe metrics), **astrometry** (WCS wrapper, the gWCS evaluator and TAN-SIP fitter under `gwcs/`, coordinate frames, grid, spectral axes, time scales, observation geometry, Gaia catalog with its disk cache, SPCC), **compose** (channel blend, LRGB, white balance, RGB drizzle), **cube** (lazy and eager cubes, moments, PV diagrams, line-fit maps), **imaging** (STF and scale, colormaps, background and DBE, deband, wavelets, deconvolution, LHE, HDRMT, curves, geometry, regions, cutouts, contours, cosmetic correction, calibration pipeline, DQ flags), **metadata** (header discovery, photometric calibration, the filter-wavelength table), **pixelmath**, **ramp** (ramp

inspection, IRS2 layout, quick slope, comparison with `_rate`), `stacking` (rejection kernel, calibration, frame cards, consistency checks, masters, CFA guard, drizzle) and `synth`.

- infra/** Infrastructure: the Arc-based image cache with the ORIG/KEY layers of the composite, configuration, FITS I/O (memory-mapped reader, MEF scanner, tile decompression, binary-table reader with array columns, the JWST ASDF extension reader, writer with multi-BITPIX output), the ASDF reader (YAML tree, blocks, zlib/bzip2/lz4) and `asdf_bridge` (header synthesis, including the fitted TAN-SIP cards), `wcs_source` (the single loader that chooses between a gWCS and the header WCS for a path), the astrometry.net client, PNG and tile rendering, binary IPC encoding and progress events.
- types/** Shared types: image references, headers, stacking and compose configuration, errors, and about 490 named constants in `types/constants.rs` (JSON keys, header card names, wizard cache keys).
- math/** Primitives: NaN-safe median/MAD, sigma clipping, SIMD log approximation (Cephes degree-8 minimax, < 1 ULP), FFT and complex arithmetic, windows, sub-pixel peak interpolation, normalization and the bounded Gauss-Newton fitter.

2.2 Non-Destructive Composite Pipeline (v0.4.5)

The composite cache uses a dual-layer architecture:

ORIG/KEY Cache Architecture

Layer	Keys	Semantics
ORIG (immutable)	<code>--composite_orig_r/g/b</code>	Written once by <code>blend/compose/RGB-FITS</code> load
KEY (working)	<code>--composite_r/g/b</code>	Reconstructed: $\text{ORIG} \times \text{WB} \rightarrow \text{KEY}$, or $\text{ORIG} \times \text{WB} \rightarrow \text{SCNR} \rightarrow \text{KEY}$

Every downstream operation (`calibrate_composite_cmd`, `apply_scnr_cmd`) reads from ORIG, applies the full transform chain, and writes to KEY. This makes WB and SCNR idempotent: applying the same factors twice produces identical results. `reset_wb_cmd` copies `ORIG` $\rightarrow$ `KEY`.

Since 0.6.4 the composite also has a processing chain of its own (`cmd::compose::composite_chain`): while the colour composite is on screen, the Processing tab's background, denoise, PSF, deconvolution, stretch, masked stretch, LHE and HDRMT steps process its three planes, a re-run of a step drops the later ones, and "Revert to original" restores the composite as it was before the first step.

2.3 Frontend (React + TypeScript)

- services/** 32 domain service modules wrapping Tauri IPC: analysis, astrometry, catalog, channelLevels, compose, compositeChain, config, contours, cosmetic, cube, cutout, db, display, export, fits, geometry (observation geometry), header, imageGeometry (rotate and flip), localContrast, pixelmath, processing, progress, pv, ramp, regions, spectral, stacking, statistics, synth, tiles, tone and toneFile. Command name strings live only in these modules; components never call `typedInvoke` directly. Arguments

cross the boundary in camelCase and arrive in Rust as `snake_case`, so a rename on either side is a runtime failure, not a compile error.

ComposeWizard

Ten steps: Channels → Stack → Align → Crop → Background → Blend → Colour → Stretch → Adjust → Export. Internal `useReducer` state with step completion tracking and suggested-next-step logic; each step declares an `enabled` predicate over the wizard data, so availability is derived from state. Since 0.6.7 the state also carries the Align run token and a stack generation counter (Section 6.4).

Processing chain

The Processing tab keeps one chain per file (`src/utils/processingChain.ts`) with fixed stages: background, denoise, deconvolution, stretch or masked stretch, LHE or HDRMT, Curves (0.6.7) and PixelMath. Re-running a step drops the later ones, and each step reads the output of the latest earlier stage. Debayer and Geometry write new files instead of joining the chain.

Preview Contexts

12 split React contexts in `PreviewContext.tsx` (`FileCtx`, `DisplayCtx`, `DoneFilesCtx`, `HistCtx`, `CubeCtx`, `RgbCtx`, `RenderCtx`, `RenderActionsCtx`, `StarOverlayCtx`, `RawPixelsCtx`, `NarrowbandCtx`, `DqCtx`), plus three composite contexts, the wizard context and a tool-host context. Splitting them this finely keeps a cursor move over the image from re-rendering the file list or the wizard.

Dock `src/components/dock/` hosts every tool window (Section 2.4).

GPU Rendering

WebGPU singleton with a WGSL fragment shader for real-time display. CPU worker fallback with identical transfer functions. Both use symmetric denominator protection and the padding threshold of the Rust backend.

2.4 Tool Windows and the Dock (v0.6.5 to v0.6.7)

Every tool is a dockable window. Four anchors take tools: Left Top and Bottom Left on the left strip, Right Top and Bottom Right on the right strip. A top anchor opens its tool as a full-height column beside the viewer; the two bottom anchors share the area under the viewer side by side, with a splitter that moves between 25 and 75 % of the width. Fourteen tools are registered (`toolRegistry.tsx`): Files, Info, Compose, Headers, Image, Astrometry, Photometry, Cube, Processing, Stacking, Synth, Export, Settings and Log.

- **Moving a tool.** Drag an icon to another group or position; a blue preview shows where the panel will open and a label reads “Move to Bottom Left” and so on. Right-click, Shift+F10 or the Menu key opens Move to, Move up, Move down and Reset layout. A moved panel keeps its contents, state and scroll position.
- **Persistence.** Groups, order, open panels and sizes persist in `localStorage` under `ab.layout.dock.v1`; splitter sizes of earlier versions are migrated, and a saved layout with the old single Analysis tool is migrated in place.
- **The five Analysis tools (0.6.6.3).** Image (histogram and STF, statistics, pixel table, regions, profiles, contours, FFT, deep zoom), Astrometry (star detection, plate solve, catalog, targets, observation geometry), Photometry (aperture photometry, the photometry table, time series), Cube (ramp, spectrum with moments and line fits, PV) and Log (the measurement log, its entry count on the icon). Detected stars are shared through a store, so the Photometry table sees the stars found in Astrometry when the two sit in different

anchors. When the selected file changes, an open Image or Cube switches to the one that fits the file (Cube for cubes and ramps) and remembers the choice per file.

- **First-run onboarding (0.6.7).** On the first launch, about 600 ms after the dock mounts, a non-modal coachmark (`role="dialog"`, `aria-modal="false"`, focus not taken) titled “Arrange your tool windows” appears at the top right of the viewer. Step 1 (“Drag vertically”) animates an icon moving to the top of a strip, with the blue drop preview filling the column and the label “Move to Right Top”; step 2 (“Drag horizontally”) moves it to the bottom of a strip with “Move to Bottom Right”. The animations are static under `prefers-reduced-motion`. Closing it (the close button, “Got it”, or Escape inside the card) or performing a real dock move writes `ab.onboarding.dock.v1 = "done"`; when storage is unavailable the card is shown once per session. The command palette entry “Show tool window tips” and the dock menu item “Show layout tips” show it again.

2.5 IPC Strategy

Binary IPC Protocol

The `get_raw_pixels_preview` command encodes a **16-byte header** followed by raw pixel data:

Offset	Type	Size	Description
0	u32	4B	Image width
4	u32	4B	Image height
8	f32	4B	Data minimum
12	f32	4B	Data maximum
16	f32[]	$W \times H \times 4B$	Raw pixel values

3 Mathematical Foundations

3.1 Screen Transfer Function (STF)

The STF implements the PixInsight-style **Midtone Transfer Function** (MTF). Given a normalized pixel value $x \in [0, 1]$ and midtone parameter $m \in (0, 1)$:

$$\text{MTF}(x, m) = \frac{(m - 1) \cdot x}{(2m - 1) \cdot x - m} \quad (1)$$

Properties:

- $\text{MTF}(0, m) = 0$ and $\text{MTF}(1, m) = 1$ for all m
- $\text{MTF}(m, m) = 0.5$ — the midtone maps to 50% brightness
- $m < 0.5$ brightens the image; $m > 0.5$ darkens it
- When $m = 0.5$: $\text{MTF}(x, 0.5) = x$ (identity)

Denominator Protection (v0.4.5)

The denominator $D = (2m - 1)x - m$ changes sign across the valid domain. The correct guard is:

$$\text{if } |D| < \varepsilon \text{ then return } x; \quad \text{if } |m - 0.5| < \varepsilon \text{ then return } x$$

The v0.4.2 GPU shader incorrectly used $\max(D, \varepsilon)$ which forces $D > 0$, inverting the transfer function for $m < 0.5$. v0.4.5 uses symmetric $|D|$ protection across GPU (WGSL), CPU worker (JS), and Rust backend, achieving pixel-identical output.

Padding pixels ($v \leq 10^{-7}$) are rendered as black across all three rendering paths, matching the Rust `is_valid_pixel` threshold.

The **auto-stretch** computes parameters from image statistics:

$$\text{shadow} = \max(0, \tilde{x}_{\text{norm}} + k \cdot \sigma_{\text{norm}}), \quad k = -2.8 \quad (2)$$

$$\text{midtone} = \text{MTF}_{\text{balance}}^{-1} \left(\frac{\tilde{x}_{\text{norm}} - \text{shadow}}{1 - \text{shadow}}, t_{\text{bg}} \right), \quad t_{\text{bg}} = 0.25 \quad (3)$$

where $\tilde{x}_{\text{norm}}$ and σ_{norm} are the normalized median and sigma ($\text{MAD} \times 1.4826$).

3.2 Asinh Stretch

$$f(x) = \frac{\text{asinh}(\alpha \cdot x)}{\text{asinh}(\alpha)} \quad (4)$$

Maps $[0, 1] \rightarrow [0, 1]$, approximately linear for small x (preserving faint detail), logarithmic compression for bright values. $\alpha \in [1, 500]$ controls aggressiveness.

3.3 Masked Stretch (v0.4.2)

Iterative MTF stretch with star mask protection. At each iteration k :

$$M^{(k)} = \text{StarMask}(I^{(k)}, \text{growth}, \text{softness}) \quad (5)$$

$$I^{(k+1)} = (1 - \alpha \cdot M^{(k)}) \cdot \text{MTF}(I^{(k)}) + \alpha \cdot M^{(k)} \cdot I^{(k)} \quad (6)$$

where α is the protection amount (default 0.85) and growth controls the mask dilation radius. Convergence when background level reaches target (default 0.25). JWST-optimized defaults: growth 0.8, protection 96%.

3.4 Cross-Correlation Alignment (Phase Correlation)

$$\text{shift} = \arg \max_{(\Delta x, \Delta y)} \mathcal{F}^{-1} \left\{ \frac{\hat{R} \cdot \hat{T}^*}{|\hat{R} \cdot \hat{T}^*|} \right\} \quad (7)$$

Hann window: $w(n) = 0.5(1 - \cos(2\pi n/N))$. Confidence: $z = (P_{\text{peak}} - \mu_P)/\sigma_P$. Coarse-to-fine: downsample to 512×512 , refine on centered crop.

3.5 Star-Based Affine Alignment (v0.4.0)

Triangle descriptor (scale-invariant):

$$T = \left(\frac{d_{\text{mid}}}{d_{\text{short}}}, \frac{d_{\text{long}}}{d_{\text{short}}} \right), \quad \text{match if } |T_r - T_t| < 0.005 \quad (8)$$

RANSAC (500 iterations) fits 6-DOF affine or 4-DOF rigid:

$$\begin{pmatrix} x_t \\ y_t \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \quad (9)$$

Sanity: offset < 25%, rotation < 5°, scale ∈ [0.85, 1.15], residual < 3 px, inlier ratio > 30%. Fallback: affine → rigid → phase correlation → identity. Warp via bicubic Catmull-Rom, parallelized per row.

3.6 Sigma-Clipped Stacking

Algorithm 1: Sigma-Clipped Stacking (median+MAD first iter, mean+std subsequent)

Input: Frames $\{F_1, \dots, F_N\}$, $\sigma_{\text{low}}, \sigma_{\text{high}}$, max iters K

Output: Stacked image S

```

1 foreach pixel ( $i, j$ ) do
2    $V \leftarrow \{F_k(i, j)\}$ , iter 0:  $\mu = \text{median}(V)$ ,  $\sigma = \text{MAD}(V) \times 1.4826$ ;
3   iters 1+:  $\mu = \text{mean}(V)$ ,  $\sigma = \text{std}(V)$ ;
4   reject  $v \notin [\mu - \sigma_{\text{low}} \cdot \sigma, \mu + \sigma_{\text{high}} \cdot \sigma]$ ;
5    $S(i, j) \leftarrow \text{mean}(V)$ ;
6 end
```

3.7 Channel Blending (v0.4.2)

Given N input channels C_k and weight matrix W :

$$\text{Out}_c = \sum_{k=1}^N W_{k,c} \cdot C_k, \quad c \in \{R, G, B\} \quad (10)$$

Preset weights (e.g., SHO: SII → R , H α → G , OIII → B) use named channelId matching with **positional fallback** (v0.4.5): when preset channelIds don't match filled bins, weights map positionally to the first N filled bins. Since 0.6.7 each channel may first be level-matched, $C_k \mapsto k_k(C_k + p_k)$ on finite non-zero pixels (Section 3.26); exact zeros are padding and contribute nothing either way.

3.8 White Balance (v0.4.0, non-destructive v0.4.5)

Reference channel selection by minimum coefficient of variation:

$$k^* = \arg \min_{k \in \{R, G, B\}} \frac{\text{MAD}_k}{\tilde{C}_k}; \quad w_j = \frac{\tilde{C}_{k^*}}{\tilde{C}_j}, \quad j \neq k^*; \quad w_{k^*} = 1 \quad (11)$$

In v0.4.5, WB is applied from ORIG cache: $\text{KEY}_c = \text{ORIG}_c \times w_c$. Idempotent.

3.9 SCNR (v0.4.0, non-destructive v0.4.5)

Two limit methods and optional luminance preservation:

$$G' = \min(G, \frac{R+B}{2}) \quad \text{or} \quad \min(G, \max(R, B)) \quad (12)$$

$$G_{\text{new}} = G + \alpha(G' - G), \quad \alpha \in [0, 1] \quad (13)$$

When luminance preservation is enabled (BT.709):

$$\Delta = \frac{0.7152 \cdot (G - G_{\text{new}})}{0.2126 + 0.0722}; \quad R' = \min(R + \Delta, 1); \quad B' = \min(B + \Delta, 1) \quad (14)$$

In v0.4.5, SCNR is applied from ORIG cache with WB pre-applied: $\text{ORIG} \times \text{WB} \rightarrow \text{SCNR} \rightarrow \text{KEY}$. Idempotent.

3.10 Drizzle Integration

$$O(i, j) = \frac{\sum_p w_p \cdot K(i - x_f^p, j - y_f^p) \cdot I_p}{\sum_p w_p \cdot K(i - x_f^p, j - y_f^p)} \quad (15)$$

Three kernels: square (top-hat), Gaussian, Lanczos-3. Flat-storage accumulator (v0.4.0): $\text{Vec}\langle\text{f32}\rangle + \text{Vec}\langle\text{u16}\rangle$ replacing $\text{Vec}\langle\text{Vec}\langle\text{f32}\rangle\rangle$. Finalize uses median/MAD sigma clipping (Stetson 1987).

3.11 Richardson-Lucy Deconvolution

$$f^{(k+1)} = f^{(k)} \cdot \left(\frac{g}{f^{(k)} * h} * \hat{h} \right) \cdot \frac{1}{1 + \lambda} \quad (16)$$

Tikhonov regularization as multiplicative damping ($\lambda = 0.001$ default). FFT-based convolution.

3.12 Background Extraction

$$B(x, y) = \sum_{i=0}^d \sum_{j=0}^{d-i} a_{ij} x^i y^j \quad (17)$$

Polynomial degree 1–5, grid 3–32, sigma-clipped sampling with `powi` exponentiation.

3.13 Wavelet Denoising

A trous wavelet decomposition: $I = \sum_{k=1}^N W_k + c_N$. Hard threshold: $W'_k = W_k$ if $|W_k| > \sigma_k t_k$, else 0. Up to 8 scales.

3.14 Calibration Pipeline (revised in v0.6.7)

$$I_{\text{cal}} = \frac{I_{\text{raw}} - M_{\text{bias}} - s \cdot M_{\text{dark}}^{(g)}}{\hat{F}} \quad (18)$$

Masters are integrated with the rejection kernel of Section 3.17 (Winsorized sigma clipping 4/3 and a mean by default). $M_{\text{dark}}^{(g)}$ is the master of the dark temperature group g nearest to the light (Section 6); it has the bias subtracted when bias frames are given. The dark scale is derived from the exposures, not typed in:

$$s = \begin{cases} \text{clamp}(t_{\text{light}}/\tilde{t}_{\text{dark}}, 0.05, 20) & \text{master bias present} \\ 1 & \text{no bias (the dark carries the bias pedestal)} \end{cases} \quad (19)$$

with $\tilde{t}_{\text{dark}}$ the median dark exposure, and s snapped to 1 when it is within 1 % of it; an unknown exposure on either side gives $s = 1$ and a warning. Exposures are read from the merged primary and science header in the order `EXPTIME`, `XPOSURE`, `EFFEXPTM`, and `EXPOSURE` only when none of `XPOSURE`, `EFFEXPTM` and `DURATION` is present (on JWST `EXPOSURE` is an exposure count). The optional dark optimisation replaces s by the value in $[0, 3]$ that minimises the robust background noise.

The flat $\hat{F}$ is built from flats calibrated by the master flat-dark when flat-darks are given ($F_i - M_{\text{flatdark}}$, no bias, no light dark); otherwise by the bias and the dark scaled to the flat exposure when both exist, or by the unscaled dark when there is no bias (with a quantified warning when the flat and dark exposures differ by more than 1 %). The calibrated flats are scaled to the first one's median, combined, and normalised to a median of 1; non-positive or non-finite pixels of the master flat become 1, and the division is skipped where $|\hat{F}| \leq 10^{-4}$. Calibration itself clamps nothing; the batch Pipeline clamps its stacked masters at zero.

3.15 Star Detection

Tile-based background (parallelized). Connected-component analysis with flux-weighted centroid:

$$\bar{x}_C = \frac{\sum_{(i,j) \in C} i \cdot (I_{ij} - b)}{\sum_{(i,j) \in C} (I_{ij} - b)} \quad (20)$$

FWHM from second intensity moments (true **median** across detected stars, v0.4.6):

$$\text{FWHM} = 2\sqrt{2 \ln 2} \cdot \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}} \quad (21)$$

3.16 SPCC: Blackbody Colour Calibration (v0.4.2, revised in v0.6.7)

Stars detected on the composite are matched to Gaia DR3 sources (through the catalog cache, $G < 17$, at most 500 rows, cone radius at least 0.01° ; a synthetic catalogue is used, and named, when fewer than three usable stars come back). Each star's BP – RP colour is mapped to an effective temperature by a piecewise-linear table (`bp_rp_to_teff`, BP – RP clamped to $[-0.5, 5]$), and its expected colour is the Planck intensity at the three channel wavelengths, normalised to their sum:

$$e_{s,c} = \frac{B_{\lambda_c}(T_s)}{\sum_{c'} B_{\lambda_{c'}}(T_s)}, \quad m_{s,c} = \frac{f_{s,c}}{\sum_{c'} f_{s,c'}}, \quad B_{\lambda}(T) = \frac{2hc^2}{\lambda^5} \frac{1}{e^{hc/\lambda kT} - 1} \quad (22)$$

with $f_{s,c}$ the aperture flux of star s in channel c . The factors are flux-weighted means of the expected-to-measured ratios, divided by the white reference W_c (a 5778 K Planck spectrum for G2V, 5500 K with a 0.98/1/1.02 tint for the average spiral, or flat for photopic) and normalised to green:

$$k_c = \frac{1}{W_c} \cdot \frac{\sum_s \sqrt{F_s} e_{s,c}/m_{s,c}}{\sum_s \sqrt{F_s}}, \quad k_c \leftarrow k_c/k_G \quad (23)$$

where F_s is the star’s total measured flux. A channel in which no matched star has flux is an error, not a neutral factor.

Wavelengths (0.6.7). The three wavelengths $\lambda_R, \lambda_G, \lambda_B$ are the effective wavelengths of the filters feeding each plane, resolved from the headers through the filter table (nominal code wavelength, e.g. F814W $\rightarrow$ 814 nm); when any of the three is unknown the defaults 640/530/460 nm are used and the result says so (`wavelength_source`). The backend accepts 300–1200 nm; the wizard offers SPCC only for broadband visible inputs (380–830 nm, no narrowband or medium-band filter). The difference matters: for a 4500 K star against a 5500 K white reference the Planck ratio $(R/G)_{4500}/(R/G)_{5500}$ is 1.2010 at 640/530 nm but 1.3743 at 814/555 nm, so with the fixed wavelengths an HST F814W/F555W/F435W field received 14 % too little red.

What the method is, and what it is not

The model is a blackbody at each channel’s effective wavelength, driven by Gaia colours. It does not integrate Gaia XP spectra through the actual filter and sensor response curves, which is what the name “spectrophotometric” implies in other tools; the wizard option therefore reads “SPCC (blackbody approximation)” since 0.6.7. Integration of XP spectra through filter curves is one of the five planned gaps (Section 16).

3.17 Pixel Rejection Families (v0.6.0)

The stacking kernel (`core/stacking/combine.rs`) separates **rejection** from **combination**. For one output pixel let $\{(v_k, c_k)\}_{k=1}^N$ be the samples contributed by the N frames, where v_k is the stored value and c_k the comparison value used for the statistics (Section 3.18); without rejection normalization $c_k = v_k$. Non-finite samples are discarded before any test. Rejection selects a surviving subset; the combination step then acts on the v_k of the survivors only, never on the c_k .

All scale estimates share one fallback: the robust scale is $1.4826 \cdot \text{MAD}$, and when the MAD is zero it degrades to 1.2533141 times the mean absolute deviation about the same centre.

Sigma clipping. Iterative, with the keep test

$$\text{keep } k \iff -\sigma_{\text{low}} s \leq c_k - \mu \leq \sigma_{\text{high}} s \quad (24)$$

On the first iteration $\mu = \text{median}(c)$ and $s = 1.4826 \cdot \text{MAD}(c)$; on every later iteration μ is the mean and s the sample standard deviation (divisor $N - 1$). The loop stops at `max_iterations` (default 5), when an iteration removes nothing, when $s \leq 0$, or when fewer than two samples remain. Defaults are $\sigma_{\text{low}} = 4.0$ and $\sigma_{\text{high}} = 3.0$.

Winsorized sigma clipping. The centre stays the median at every iteration; only the scale changes. With $s_0 = 1.4826 \cdot \text{MAD}(c)$ and cutoff κ (default 5.0), the samples are first Winsorized,

$$\tilde{c}_k = \text{clamp}(c_k, \mu - \kappa s_0, \mu + \kappa s_0), \quad (25)$$

and a fixed-point iteration then re-estimates the scale with the tails held at ± 1.5 current sigmas:

$$u_k^{(j)} = \text{clamp}(\tilde{c}_k, \mu - 1.5 s^{(j)}, \mu + 1.5 s^{(j)}) \quad (26)$$

$$s^{(j+1)} = 1.134 \cdot \text{stdev}(u^{(j)}) \quad (27)$$

Tails are replaced, not discarded

Every sample outside the window is *set to the window edge* and stays in the sum; the divisor remains N (variance uses $N - 1$). The estimator therefore keeps the full sample count while the outliers lose their leverage, which is what distinguishes it from trimming. The factor 1.134 restores consistency with the Gaussian standard deviation after that replacement. At most 10 steps, stopping when the relative change falls below 10^{-4} .

The keep test is then Equation 24 with μ the median and s the Winsorized sigma. Needs at least three samples.

Linear fit clipping. The samples are sorted by c and a straight line in *rank* is fitted by least squares over the inner half of the ranks, that is $i \in [\lfloor n/4 \rfloor, n - \lfloor n/4 \rfloor]$:

$$c_{(i)} \approx a + b i, \quad r_i = c_{(i)} - (a + b i), \quad d = 1.4826 \cdot \text{median}_i |r_i| \quad (28)$$

Residuals are evaluated at *all* ranks, not only the fitted ones, and the keep test is $-\sigma_{\text{low}} d \leq r_i \leq \sigma_{\text{high}} d$. The method needs at least 5 samples; below that the kernel falls back to plain sigma clipping.

Percentile clipping. A single non-iterative pass about the median $m = \text{median}(c)$, with bounds expressed as fractions of the median magnitude:

$$\text{keep } k \iff -p_{\text{low}} |m| \leq c_k - m \leq p_{\text{high}} |m| \quad (29)$$

Defaults $p_{\text{low}} = 0.2$ and $p_{\text{high}} = 0.1$. Needs at least 3 samples and a finite, non-zero $|m|$; otherwise the set passes untouched.

Min/max clipping. No statistics at all: sort by c , drop the n_{low} smallest and the n_{high} largest (both default 1). Skipped entirely when $N \leq n_{\text{low}} + n_{\text{high}}$.

Method	Criterion	Iterative	Min. N
Sigma clip	deviation from median/mean in units of s	yes	2
Winsorized sigma clip	same, s from tail replacement	yes	3
Linear fit clip	residual from a line fitted in rank	yes	5
Percentile clip	deviation as a fraction of $ m $	no	3
Min/max	fixed count from each end	no	$n_{\text{low}} + n_{\text{high}} + 1$

Table 1: Rejection families and their guards (`RejectionParams`).

The combination step. Applied to the surviving v_k :

$$S = \begin{cases} \frac{1}{K} \sum_k v_k & \text{mean, unweighted} \\ \frac{\sum_k w_{f(k)} v_k}{\sum_k w_{f(k)}} & \text{mean, per-frame weights } w \\ \text{median}_k v_k, \quad \min_k v_k, \quad \max_k v_k & \text{median, min, max} \end{cases} \quad (30)$$

where $f(k)$ is the frame index of sample k . Weights act on the mean only; the weighted form reverts to the plain mean when $\sum w \leq 10^{-12}$. If every sample is rejected the pixel takes the last centre used by the rejection loop, or 0 when no finite centre was computed. With `rejection_maps` enabled the kernel also returns two `u16` maps holding the low and high rejection counts per pixel.

3.18 Frame Normalization (v0.6.0)

Normalization brings frames onto a common photometric footing before rejection. The per-frame statistics are computed on the **mutual overlap**: a linear pixel index contributes only when the value is finite in *every* frame, so all frames are described over exactly the same set of pixels. The index is strided so that at most 131 072 positions are sampled. For each frame the location is the median and the scale is $1.4826 \cdot \text{MAD}$ over that set; frame 0 is the reference (m_0, s_0).

Each frame f receives an affine map $v \mapsto k_f v + z_f$ with

$$(z_f, k_f) = \begin{cases} (0, 1) & \text{none} \\ (m_0 - m_f, 1) & \text{additive} \\ (0, m_0/m_f) & \text{multiplicative} \\ \left(m_0 - \frac{s_0}{s_f} m_f, \frac{s_0}{s_f}\right) & \text{additive with scaling (default)} \\ (0, s_0/s_f) & \text{multiplicative with scaling} \end{cases} \quad (31)$$

The plain forms equalise the sky level only; the scaling variants also equalise the dispersion, so that frames taken under different transparency or sky brightness contribute noise of comparable amplitude. Both ratios are guarded: s_0/s_f becomes 1 when either scale is non-positive or non-finite, and m_0/m_f becomes 1 when $|m_f| \leq 10^{-12}$.

Scale + offset rejection normalization is a separate, non-destructive stage. After the maps above have been applied, the post-normalization statistics are $m'_f = k_f m_f + z_f$ and $s'_f = k_f s_f$, and each frame gets a relative scale $\rho_f = s'_f/s'_0$. The rejection kernel then sees a comparison value

$$c_k = \frac{v_k - m'_f}{\rho_f} + m'_0 \quad (32)$$

while the value entering the combination remains v_k . Rejection decisions are therefore taken in the reference frame's units, but the stacked result is still built from the frames' own pixels. A frame with $m'_f = m'_0$ and $\rho_f = 1$ is left alone. Statistics are computed whenever either normalization or scale + offset rejection normalization is requested.

Noise weights under normalization (0.6.7). The $1/\sigma^2$ noise weights are measured on the raw frames, but the frames are combined after the map $v \mapsto k_f v + z_f$, which multiplies each frame’s noise by k_f . The weights that enter the mean are therefore

$$\boxed{w'_f = \frac{w_f}{k_f^2}} \quad (33)$$

so that a frame dimmed by cloud to half its signal (raw σ halved, $k_f = 2$) no longer receives four times the weight of a clear frame of equal true noise. With two such frames the old weighting gave a stack noise of 0.825 against the optimal $1/\sqrt{2} = 0.707$. The response reports the effective weights per input (`weights_applied`, `null` for excluded frames), and the Stack tab hint shows their range renormalised to mean 1 together with the number of frames it describes. Median combination ignores weights, as before.

3.19 Thin-Plate Spline Background Extraction — DBE (v0.6.0)

The Spline background model (`core/imaging/dbe.rs`) fits a **regularised thin-plate spline** through box samples, in place of the global polynomial surface of the Background Extraction subsection. Coordinates are divided by $S = \max(\text{rows}, \text{cols})$ before the fit, so the node spacing is $O(1)$.

The radial basis function is the two-dimensional thin-plate kernel

$$\boxed{\varphi(r) = r^2 \ln r \quad (r > 0), \quad \varphi(0) = 0} \quad (34)$$

and the model is that kernel plus an affine term,

$$B(x, y) = a_0 + a_1 x + a_2 y + \sum_{j=1}^n \lambda_j \varphi(\|\mathbf{p} - \mathbf{p}_j\|) \quad (35)$$

The coefficients come from one $(n+3) \times (n+3)$ system solved by Gaussian elimination with partial pivoting:

$$\begin{pmatrix} \mathbf{K} + \mu \mathbf{W}^{-1} & \mathbf{P} \\ \mathbf{P}^\top & \mathbf{0} \end{pmatrix} \begin{pmatrix} \boldsymbol{\lambda} \\ \mathbf{a} \end{pmatrix} = \begin{pmatrix} \mathbf{z} \\ \mathbf{0} \end{pmatrix}, \quad K_{ij} = \varphi(\|\mathbf{p}_i - \mathbf{p}_j\|), \quad \mathbf{P} = \begin{pmatrix} 1 & x_1 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & x_n & y_n \end{pmatrix} \quad (36)$$

The three zero rows are the usual side conditions $\sum_j \lambda_j = \sum_j \lambda_j x_j = \sum_j \lambda_j y_j = 0$, which make the affine part unique. Regularisation is the diagonal term

$$(\mu \mathbf{W}^{-1})_{ii} = \frac{\mu}{\max(w_i, 10^{-6})}, \quad \mu = \text{smoothing} \cdot \begin{cases} \bar{d}^2 |\ln \bar{d}| & 0 < \bar{d} < 1 \\ 1 & \text{otherwise} \end{cases} \quad (37)$$

where $\bar{d}$ is the mean nearest-neighbour distance between scaled nodes, so the penalty is scaled by the kernel’s own magnitude at the typical node spacing. Smoothing defaults to 0.25. Because μ is divided by the sample weight w_i , a poorly filled sample box is smoothed harder than a fully covered one; at $\mu = 0$ the spline interpolates the samples exactly.

Sample placement. Manual samples (Point regions) are taken first. The automatic grid divides the image into $g \times g$ cells (g default 10, at least 2, each cell at least 4 px per axis), trims a margin

of a quarter cell on each side and places the sample at the centre of the remaining inner box. Automatic positions within 1 px of a manual one are dropped, and the automatic set is thinned uniformly so that the total does not exceed `max_samples` (default 400).

Each sample is a box of half-width `sample_radius` (default 5, so 11×11 px) reduced by sigma-clipped statistics ($\kappa = 2.5$, 3 iterations) to a median, a sigma and a maximum; the sample value is the median and its weight is the finite fraction of the nominal box area, capped at 1.

Sample rejection has four recorded reasons:

- **bounds** — the rounded centre falls outside the image
- **finite** — fewer than 25% of the nominal box pixels are finite
- **star** — the box maximum exceeds $\text{median} + 5 \max(\sigma_{\text{sky}}, \sigma_{\text{box}})$, where σ_{sky} is the median of the usable per-box sigmas
- **deviation** — after a first fit, the residual exceeds $\max(\text{tolerance} \cdot \sigma_{\text{sky}}, 3\sigma_{\text{res}})$, with σ_{res} the MAD-based sigma of the residuals and tolerance default 1.0

Deviation rejection runs once and the spline is refitted on the survivors; it is skipped when it would leave fewer than 4 accepted samples, which is also the minimum for any fit. The final model is evaluated on a coarse lattice of step $\max(8, \min(\text{rows}, \text{cols})/256)$ and bilinearly interpolated to full resolution, then applied by subtraction or by division (guarded at $B > 10^{-10}$). The reported RMS residual is taken over the accepted samples only.

3.20 Aperture Photometry with Fractional Pixel Weights (v0.6.0)

`core/analysis/aperture.rs` replaces the centre-in-circle rule with a sub-pixel coverage weight. For a pixel whose centre is at distance d from the aperture centre, with $\delta = 1/\sqrt{2}$ the half-diagonal of a pixel:

$$w = \begin{cases} 1 & d + \delta \leq r \\ 0 & d - \delta > r \\ \frac{1}{n^2} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{1}[\|\mathbf{s}_{ij} - \mathbf{c}\|^2 \leq r^2] & \text{otherwise} \end{cases} \quad (38)$$

where the sampling points are $\mathbf{s}_{ij} = \mathbf{p} + \left(\frac{i+0.5}{n} - \frac{1}{2}, \frac{j+0.5}{n} - \frac{1}{2}\right)$, an $n \times n$ grid over the pixel with $n = \text{subsamples}$, default 5. Only the boundary band is sampled; interior and exterior pixels take the closed forms. An annulus weight is the difference of the two circle fractions, $w_{\text{out}} - w_{\text{in}}$, evaluated on the same grid.

The aperture integral and its effective area are

$$F = \sum_p w_p I_p, \quad A = \sum_p w_p \quad (39)$$

over pixels that are finite and not set in the exclusion mask; masked and non-finite pixels are counted separately and reported. A is what `aperture_pixels` rounds, so it tracks πr^2 rather than the count of partially covered pixels.

Local background. The sky annulus has radii $2r_{\text{ap}}$ and $3r_{\text{ap}}$ about the refined centroid. Only annulus pixels with $w \geq 0.5$ are taken as members. From the finite values in that set, $\bar{b}$ is the mean of the interquartile range (ranks $\lfloor n/4 \rfloor$ to $\lfloor 3n/4 \rfloor$) and $\sigma_{\text{sky}} = 1.4826 \cdot \text{median}_i |b_i - \text{median}(b)|$

over the whole set. The net flux is $F_{\text{net}} = F - \bar{b}A$ and the instrumental magnitude $m = -2.5 \log_{10} F_{\text{net}}$.

The aperture radius defaults to $1.5 \times \text{FWHM}$ clamped to $[2, 60]$ px, with the FWHM taken from second moments truncated at half maximum and corrected by the factor $1 - \ln 2$ for that truncation.

Error propagation always carries the uncertainty of the sky level itself,

$$\sigma_b^2 = \frac{A^2 \sigma_{\text{sky}}^2}{n_{\text{bg}}}, \quad (40)$$

and then takes one of two forms depending on whether an ERR plane accompanies the data:

$$\sigma_F^2 = \sum_p (w_p e_p)^2 + \sigma_b^2 \quad \text{ERR plane present} \quad (41)$$

$$\sigma_F^2 = \frac{\max(F_{\text{net}}, 0)}{g} + A \sigma_{\text{sky}}^2 + \sigma_b^2 \quad \text{no ERR plane} \quad (42)$$

In Equation 41 each pixel's quoted error is scaled by the same fractional weight used for the flux. In Equation 42 the Poisson term is present only when a gain g in electrons per ADU is supplied, and is zero otherwise; the second term is the shot noise of the sky within the aperture. The result reports which of the two was used, and $\text{SNR} = F_{\text{net}}/\sigma_F$. A pixel whose ERR value is not finite falls back to the sky-and-gain noise model. Since 0.6.7 the gain field is prefilled from a header-derived model (Section 3.25); the backend still uses only the gain the request carries, and reports it as `gain_used`.

Curve-of-growth aperture correction. The cumulative net flux is measured at integer radii out to $4r_{\text{ap}}$,

$$C(r) = \sum_{p \in \mathcal{A}(r)} w_p I_p - \bar{b} \sum_{p \in \mathcal{A}(r)} w_p \quad (43)$$

and the plateau is the first radius $r \geq \max(\lfloor r_{\text{ap}} \rfloor + 1, 3)$ satisfying

$$C(r) > 0 \quad \text{and} \quad |C(r) - C(r-2)| < 0.01 C(r) \quad (44)$$

that is, the growth curve has flattened to better than 1% over two radial steps. If that radius reaches the inner edge of the sky annulus, $2r_{\text{ap}}$, no correction is reported, because the curve would then be contaminated by the region used to define the background. Otherwise the correction factor is $F_{\text{net}}/C(r_{\text{plateau}})$ and $C(r_{\text{plateau}})$ is reported as the total flux.

3.21 Barycentric and Heliocentric Radial Velocity (v0.6.0)

`core/astrometry/spectral.rs` computes the correction from the header alone, using the **Meeus low-precision ephemeris** (*Astronomical Algorithms*, ch. 25 for the Sun and a truncated ch. 47 for the Moon). Times come from `core/astrometry/time.rs`: Julian dates through the Meeus calendar algorithm, T in Julian centuries from J2000.0, and GMST from

$$\theta_{\text{GMST}} = 280.46061837 + 360.98564736629 d + 0.000387933 T^2 - \frac{T^3}{38\,710\,000} \quad (45)$$

with $d = \text{JD} - 2451545.0$. UTC is treated as UT1 and $\text{TT} = \text{UTC} + 69.2\text{ s}$.

Solar coordinates.

$$L_0 = 280.46646 + 36000.76983 T + 0.0003032 T^2 \quad (46)$$

$$M = 357.52911 + 35999.05029 T - 0.0001537 T^2 \quad (47)$$

$$e = 0.016708634 - 0.000042037 T - 0.0000001267 T^2 \quad (48)$$

$$C = (1.914602 - 0.004817 T - 0.000014 T^2) \sin M \\ + (0.019993 - 0.000101 T) \sin 2M + 0.000289 \sin 3M \quad (49)$$

$$\odot = L_0 + C, \quad R = \frac{1.000001018 (1 - e^2)}{1 + e \cos(M + C)} \quad (50)$$

The Earth–Moon barycentre sits at heliocentric ecliptic longitude $\odot + 180^\circ$ and radius R ; the Earth itself is offset from it by the geocentric lunar vector divided by $1 + 81.30056$. The heliocentric velocity is a central difference of that position with $\Delta t = 1/48$ day, converted with $1 \text{ AU} = 149\,597\,870.7 \text{ km}$, then rotated from the ecliptic of date to equatorial J2000 by $R_z(-(1.3969713 T + 0.0003086 T^2))$ followed by $R_x(23.4392911^\circ)$.

Diurnal term. On the WGS84 ellipsoid ($a = 6\,378\,137 \text{ m}$, $f = 1/298.257223563$, $e^2 = 2f - f^2$), with $N = a/\sqrt{1 - e^2 \sin^2 \phi}$ the prime vertical radius, the site's distance from the rotation axis and its velocity are

$$\rho = (N + h) \cos \phi, \quad \mathbf{v}_{\text{rot}} = \omega \rho (-\sin \theta_{\text{LST}}, \cos \theta_{\text{LST}}, 0), \quad \omega = 7.292115 \times 10^{-5} \text{ rad s}^{-1} \quad (51)$$

with $\theta_{\text{LST}} = \theta_{\text{GMST}} + \lambda$ and longitude read east-positive.

Projection. With the target unit vector $\hat{\mathbf{u}} = (\cos \delta \cos \alpha, \cos \delta \sin \alpha, \sin \delta)$,

$$v_{\text{helio}} = \mathbf{v}_{\oplus} \cdot \hat{\mathbf{u}} + \mathbf{v}_{\text{rot}} \cdot \hat{\mathbf{u}} \quad (52)$$

$$v_{\text{bary}} = v_{\text{helio}} + \mathbf{v}_{\odot} \cdot \hat{\mathbf{u}} \quad (53)$$

where $\mathbf{v}_{\odot} = -\sum_p \mathbf{v}_p m_p / M_{\odot}$ is the Sun's reflex motion about the solar-system barycentre, modelled as circular orbits for Jupiter, Saturn, Uranus and Neptune. The sign convention is that the correction is *added* to a topocentric velocity, positive when the observer moves toward the target.

Quoted accuracy

The correction is reported with `accuracy_kms` = 0.02 km s^{-1} and the method string *Meeus low-precision ephemeris*. A SPECSYS of GEOCENTR skips the diurnal term; frames already barycentric or beyond report a zero shift with the frame named; spacecraft headers (TELESCOP containing JWST, HST or ROMAN) take the correction as $-\text{VELOSYS}/1000$ in km s^{-1} from the header, or are refused with the reason, since the spacecraft ephemeris is not reconstructible from the header.

Air and vacuum wavelengths. Following Greisen et al. (2006), FITS Paper III eq. 65, with λ in micrometres,

$$n(\lambda) = 1 + 10^{-6} \left(287.6155 + \frac{1.62887}{\lambda^2} + \frac{0.01360}{\lambda^4} \right) \quad (54)$$

The forward direction is direct, $\lambda_{\text{air}} = \lambda_{\text{vac}}/n(\lambda_{\text{vac}})$. The inverse has no closed form and is solved by fixed-point iteration $\lambda^{(k+1)} = \lambda_{\text{air}} n(\lambda^{(k)})$ started at $\lambda_{\text{air}} n(\lambda_{\text{air}})$, converging at $10^{-10} \mu\text{m}$ within at most 50 steps. Below $0.2 \mu\text{m}$ the formula is outside its stated range and the wavelength is passed through unchanged, with a note.

Velocity conventions. With $c = 299\,792.458 \text{ km s}^{-1}$, observed λ and rest λ_0 :

$$v_{\text{opt}} = c \frac{\lambda - \lambda_0}{\lambda_0} \quad \lambda = \lambda_0(1 + \beta) \quad (55)$$

$$v_{\text{rad}} = c \frac{\lambda - \lambda_0}{\lambda} \quad \lambda = \frac{\lambda_0}{1 - \beta} \quad (56)$$

$$v_{\text{rel}} = c \frac{\lambda^2 - \lambda_0^2}{\lambda^2 + \lambda_0^2} \quad \lambda = \lambda_0 \sqrt{\frac{1 + \beta}{1 - \beta}} \quad (57)$$

with $\beta = v/c$; the right-hand column is the inversion used to place a velocity back on the wavelength axis. The three agree to first order in β and diverge beyond it.

3.22 Moment Maps (v0.6.0)

`core/cube/moments.rs` collapses a spectral cube over a channel range $[z_0, z_1]$ into three maps. The spectral axis is first converted to velocity by the convention of Section 3.21; an axis already stored in velocity units is used as-is. Channel widths come from a centred difference,

$$\Delta v_z = \begin{cases} |v_1 - v_0| & z = 0 \\ |v_{n-1} - v_{n-2}| & z = n - 1 \\ \frac{1}{2} |v_{z+1} - v_{z-1}| & \text{otherwise} \end{cases} \quad (58)$$

Per-pixel continuum. Two side windows $[z_a, z_b]$ and $[z_c, z_d]$ are merged, sorted and deduplicated. When the two windows differ and at least three channels remain, a first-order baseline is fitted per pixel by ordinary least squares in the channel index; otherwise the fit collapses to a constant, and it also collapses when fewer than three channels are finite or the normal equations are degenerate:

$$b_i(z) = \alpha_i + \beta_i z, \quad \beta_i = \frac{n \sum zy - \sum z \sum y}{n \sum z^2 - (\sum z)^2}, \quad \alpha_i = \frac{\sum y - \beta_i \sum z}{n} \quad (59)$$

The per-pixel noise σ_i is the MAD-based robust sigma of that fit's residuals, $1.4826 \cdot \text{MAD}$. With no windows the raw channel values are used and a single global sigma is taken from a line-free channel, $z_0 - 1$ if it exists, otherwise $z_1 + 1$.

The moments. With the continuum-subtracted sample $s_{i,z} = I_{i,z} - b_i(z)$:

$$M_0 = \sum_{z=z_0}^{z_1} s_{i,z} \Delta v_z \quad (60)$$

$$M_1 = \frac{\sum_{z:s>0} s_{i,z} v_z}{\sum_{z:s>0} s_{i,z}} \quad (61)$$

$$M_2 = \sqrt{\max\left(\frac{\sum_{z:s>0} s_{i,z} v_z^2}{\sum_{z:s>0} s_{i,z}} - M_1^2, 0\right)} \quad (62)$$

M_0 is the integrated intensity and takes every finite channel, positive or negative, weighted by its own channel width. M_1 and M_2 are the intensity-weighted mean velocity and its dispersion, and both restrict the sum to *positive* samples so that negative noise excursions cannot drive the weights or push the variance negative; the variance is clamped at zero before the root regardless.

SNR masking. The noise used for a pixel is the larger of its own residual sigma and a global floor, the map median of the per-pixel sigmas. A pixel passes when

$$\boxed{\frac{M_0}{\sigma_i \sqrt{N_{\text{ch}} \Delta v}} \geq t} \quad (63)$$

with $N_{\text{ch}} = z_1 - z_0 + 1$, $\overline{\Delta v}$ the mean channel width over the range and t the threshold, default 3.0. The $\sqrt{N_{\text{ch}} \Delta v}$ factor is the propagated noise of the M_0 sum itself. M_1 and M_2 are written only where the test passes; M_0 is kept for failing pixels when `mask_below_threshold` is disabled, and stays NaN otherwise. When no noise estimate is available the pixel is passed and counted as unmaskable.

3.23 Wavelet Per-Scale Detail Bias (v0.6.0)

`core/imaging/wavelet.rs` implements the à trous (“with holes”) transform with the B_3 spline kernel

$$h = \frac{1}{16} [1, 4, 6, 4, 1] \quad (64)$$

applied separably, with a hole spacing of 2^k taps at scale k , edge clamping, and renormalisation over the finite taps only so that NaN pixels do not spread. Starting from $c_0 = I$:

$$c_{k+1} = h_k * c_k, \quad w_{k+1} = c_k - c_{k+1}, \quad I = \sum_{k=1}^N w_k + c_N \quad (65)$$

Up to 8 scales, default 5. The decomposition is exact: the layers and the residual sum back to the input.

Per-scale threshold. The noise is estimated once from the finest layer, $\sigma_{w_1} = 1.4826 \cdot \text{median}|w_1|$, and referred back to the image by the B_3 scaling table

$$\sigma = \frac{\sigma_{w_1}}{\kappa_0}, \quad \kappa = (0.8908, 0.2007, 0.0856, 0.0413, 0.0205, 0.0103, 0.0051), \quad \kappa_k = \frac{\kappa_6}{2^{k-6}} \quad (k > 6) \quad (66)$$

so that scale k is thresholded at its own expected noise amplitude:

$$\boxed{T_k = t_k \sigma \kappa_k} \quad (67)$$

with per-scale multipliers t defaulting to (3, 2.5, 2, 1.5, 1); scales beyond the supplied list reuse the last entry. Thresholding is soft by default, $w' = \text{sgn}(w) \max(|w| - T_k, 0)$, or hard, $w' = w \cdot \mathbf{1}[|w| > T_k]$.

Detail bias. Reconstruction applies a per-layer gain rather than summing the layers back at unit weight:

$$I' = c_N + \sum_{k=1}^N (1 + \beta_k) w'_k \quad (68)$$

The bias β_k is exposed on $[-1, +3]$ in steps of 0.05, default 0, and layers with no entry take $\beta_k = 0$. At $\beta_k = 0$ the layer is restored as it stands, so a run with all biases zero and all thresholds zero is the identity. At $\beta_k = -1$ the gain vanishes and the scale is removed from the image entirely; positive values amplify structure at that scale, up to four times its measured amplitude at $\beta_k = 3$. Because each β_k multiplies one layer of an exact decomposition, the operation is a linear per-scale filter and does not move energy between scales.

3.24 Local Histogram Equalization and HDR Multiscale Transform (v0.6.0)

LHE (`core/imaging/local_contrast.rs`) is contrast-limited adaptive histogram equalization. The image is covered by a grid of tiles of side $R = \text{kernel_radius}$, clamped to $[16, 512]$ with default 64, and a lookup table is built at each tile centre from the pixels within R of it — a disc when `circular` is set, the bounding square otherwise. Values are mapped onto $B = 2^b$ bins, $b \in \{8, 10, 12\}$ with default 8, by

$$p(v) = \text{clamp}\left(\frac{v - v_{\min}}{v_{\max} - v_{\min}}, 0, 1\right) (B - 1), \quad j = \lfloor p(v) + \frac{1}{2} \rfloor \quad (69)$$

over the finite range of the whole image.

Clip limit and redistribution. With n pixels in the window and contrast limit γ clamped to $[1, 64]$ (default 2.0),

$$c = \max\left(\frac{\gamma n}{B}, 1\right), \quad E = \sum_{j=0}^{B-1} \max(h_j - c, 0), \quad p_j = \min(h_j, c) + \frac{E}{B} \quad (70)$$

The clip limit is therefore a multiple of the *mean* bin population: $\gamma = 1$ flattens the histogram to uniform and gives no local contrast gain, while larger values allow progressively steeper transfer curves. The clipped mass E is not discarded but spread uniformly over all B bins, so $\sum_j p_j = n$ exactly and the transfer function remains a valid CDF. The lookup table is the mid-point corrected cumulative sum

$$L_j = \text{clamp}\left(\frac{1}{n} \left(\sum_{i \leq j} p_i - \frac{1}{2} p_j\right), 0, 1\right) \quad (71)$$

Each output pixel is interpolated linearly between the two adjacent bins of a table and bilinearly between the four surrounding tile tables, then blended with the input by the amount $a \in [0, 1]$ and clamped back into the input range:

$$v' = \text{clamp}((1 - a)v + a\hat{v}, v_{\min}, v_{\max}) \quad (72)$$

Non-finite pixels are passed through, and an image whose finite range is below 10^{-12} is returned unchanged.

HDRMT (`core/imaging/hdr.rs`) compresses large-scale dynamic range through the same à trous decomposition. Each of `iterations` passes (1–4, default 1) does:

1. decompose the working image into N detail layers (2–8, default 6) plus the residual c_N ;
2. clamp the residual from above at its own median, $c_N \leftarrow \min(c_N, \text{median}(c_N))$;
3. reconstruct with the per-layer bias of Section 3.23, $\beta_k = -\text{overdrive}$ for the two coarsest detail layers ($k + 2 \geq N$) and $\beta_k = 0$ elsewhere, with overdrive clamped to $[0, 1]$;
4. rescale the result to $[0, 1]$.

Step 2 is the transform itself: every pixel whose large-scale background sits above the median is pulled down to it, so a bright core and a faint outskirts end up on the same footing, while the detail layers that carry the structure are reinjected untouched. Optional inversion runs the whole pass on $1 - v$ to compress dark rather than bright structure; optional deringing blends the original normalised image back under a star mask with weight $m \cdot a$. The image is finally mapped back to its original range. On RGB the transform runs on the BT.709 luminance ($0.2126R + 0.7152G + 0.0722B$) and the ratio is applied to the three channels, so hue is preserved.

Why linear input is refused

HDRMT rejects any image whose finite median is below 1% of its finite maximum, with the message “*HDRMT expects a non-linear (stretched) image; median is below 1% of the maximum*”. On linear astronomical data almost every pixel sits just above the sky floor while the stars span several decades, so the residual median of step 2 would lie at that floor and the clamp would flatten essentially the whole frame. The transform assumes a stretch has already redistributed the histogram, which is why it is placed after the STF in the pipeline rather than before it.

3.25 Effective Gain for Photometry (v0.6.7)

Without an ERR plane, the Poisson term of Equation 42 needs a gain g in electrons per data unit. `core/analysis/gain.rs` derives it from the header and the HDU list; the panels prefill the field with it and say where it came from, and a value the user typed survives a file change. The model has three steps.

Unit class. The normalised BUNIT selects how a gain applies:

Class	BUNIT	Unit gain g_u
counts	COUNTS, COUNT, DN, ADU, or none	header gain
count rate	COUNTS/S, COUNT/S, DN/S, ADU/S	header gain $\times$ EXPTIME
electrons	ELECTRONS, ELECTRON, E-	1
electron rate	ELECTRONS/S, ELECTRON/S, E-/S	EXPTIME
calibrated	MJy/sr, Jy, MJy	none
other	anything else (ERG/S/CM2/A, unknown text)	none

Table 2: Unit classes of the gain model. The count-rate class is a whitelist: a test for “/S” alone would also match ERG/S/CM2/A.

The header gain is the first finite positive value of **EGAIN** (MaxIm DL, N.I.N.A., SGP, SharpCap,

ASIAIR), ATODGAIN (HST WFPC2, STIS), CCDGAIN (HST ACS, WFC3, STIS) or ADCGAIN (NICMOS). GAIN is never applied, because ZWO, QHY and ASCOM drivers write the camera gain setting there; when it is the only gain card the caption says it is present and not applied, so a user of IRAF-style headers, where GAIN is in e^-/ADU , can type it.

Combination. A stack written by AstroBurst carries $N = \text{NCOMBINE}$, the combine method ABCOMB and, for drizzle, the output scale $s = \text{ABDRZSCL}$ (Section 6). The effective gain is

$$g_{\text{eff}} = \begin{cases} N g_u & \text{mean} \\ 2N g_u / \pi & \text{median (its variance is } \pi/2 \text{ that of the mean)} \\ g_u & \text{min or max (an extreme-value combine has no Poisson scaling), with a note} \\ N g_u / s^2 & \text{drizzle, with a note that output pixels are correlated} \\ g_u & \text{NCOMBINE without ABCOMB, with a note suggesting } N g_u \text{ for a mean stack} \end{cases} \quad (73)$$

The last case covers external files: the WFPC2 mosaics that ship with the app carry $\text{NCOMBINE}=2$ and $\text{ATODGAIN}=7$ with no combine card, so the model prefills 7 and the note suggests 14 for a mean stack.

Route. When the file has an ERR extension of the image’s size, the ERR plane is the noise model whatever the unit, and a header gain is reported only as the fallback for non-finite ERR pixels (it is not prefilled); JWST i2d (MJy/sr) and NIRSpec _rate (DN/s) files both take this route. A display-referred image (ABDISP=T) has no gain. The gain and combine cards are dropped from rescaled and display-referred outputs, so a PixelMath result in $[0, 1]$ cannot inherit EGAIN.

Why it matters. A CMOS frame with EGAIN=0.25, a star of net 40 000 ADU in a 100 px aperture, sky $\sigma = 8 \text{ ADU}$ and a 300 px annulus gives, without a gain, $\sigma_F = \sqrt{100 \cdot 64 + 100^2 \cdot 64/300} = 92.4 \text{ ADU}$ and SNR 433; with the gain the Poisson term adds 40 000/0.25 and $\sigma_F = 410.5 \text{ ADU}$, SNR 97. For a 16-frame mean stack of such frames, EGAIN alone would give $\sigma_F = 410.5$ while the effective gain of 4 gives 136. When no gain applies, click, batch and time-series photometry return the warning “flux errors leave out source photon noise: no gain was given and the frames carry no ERR plane; enter the gain in e^-/ADU to include it” for count units, or the model’s note for calibrated and physical units.

3.26 Level Matching and the Composite STF Link (v0.6.7)

Match levels (Compose > Blend, on by default when a narrowband filter is loaded) equalises the background and the diffuse-signal spread of the input channels before the blend. For each used channel c , `measure_channel_levels_cmd` returns the median m_c and the MAD d_c of its finite non-zero pixels (histogram-sampled above 4 Mpx). The reference is the channel of largest MAD, $r = \arg \max_c d_c$, and

$$k_c = \frac{d_r}{d_c} \geq 1, \quad p_c = \frac{m_r}{k_c} - m_c, \quad v \mapsto k_c(v + p_c) = k_c v + z_c, \quad z_c = m_r - k_c m_c \quad (74)$$

so every matched channel has median m_r and MAD d_r . The blend weights of channel c are multiplied by k_c and its pedestal p_c is added in the backend’s per-pixel loop to finite non-zero pixels only. A channel with $d_c \leq 0$ or a non-finite MAD, or a scale above 10^4 , is refused with a message. The scales go into the “Level match:” line under Blend and into one FITS HISTORY line per channel, `Level match {ch}: x{k} {±z}`.

Why median and MAD. The 0.6.5.0 rule scaled each channel by the largest ($p_{99.5} - p_{50}$) over its own. On the JWST NIRCам trio of the Pillars (F090W, F187N, F200W in MJy/sr; medians 2.589, 40.71, 5.188; MADs 0.3058, 6.363, 0.6649) the 99.5th percentile of F200W is dominated by stars and that of F187N by diffuse Pa α , so the old scales (1.379, 1, 14.875) left the linked preview 68 % green and 31 % blue. The MAD of these channels measures the spread of the diffuse nebula (on a 1500×1500 F090W block the MAD is 0.147 while the pixel-noise MAD is 0.031), which is what should be balanced. With the median/MAD rule the reference is F187N, the scales are 9.570 (F200W), 1 and 20.81 (F090W) and the pedestals in output units -8.94 , 0 and -13.17 ; the preview is 81 % neutral, and the exported FITS planes have equal medians and MADs instead of a $15\times$ level imbalance.

STF link rule. The preview stretch of the blended composite is decided on the output planes, whether Match levels is on or off, because the blend matrix need not be diagonal (bicolour and wavelength-spreading presets mix channels). With m_R, m_G, m_B the plane medians and $\sigma_R, \sigma_G, \sigma_B$ their MAD-based widths,

$$\text{unlinked} \iff \max_c m_c - \min_c m_c > 3 \max_c \sigma_c \vee \left(\min_c m_c > 0 \wedge \frac{\max_c m_c}{\min_c m_c} > 1.5 \right) \quad (75)$$

A level-matched diagonal blend has equal medians and stays linked; background-subtracted inputs have medians near zero and stay linked. An unlinked stretch applies to the preview, the Adjust step and the PNG export; the composite FITS data are not changed. SPCC factors computed after Match levels are divided by the scale of the channel feeding each plane.

3.27 Gaussian Line-Fit Maps (v0.6.6)

`core/cube/linefit.rs` fits every spaxel of an IFU cube over a brushed line window of 5 to 400 channels. The continuum is fitted first from the two side windows and then held fixed: linear in the channel index x when both windows have at least 3 usable channels, constant otherwise (mask bit 2). The line is a Gaussian in channel space,

$$y(x) = a_0 + a_1(x - x_{\text{ref}}) + A \exp\left(-\frac{(x - x_c)^2}{2\sigma_x^2}\right) \quad (76)$$

fitted by the damped, bounded Gauss-Newton fitter of `math::gauss_newton` with weights $1/\text{ERR}^2$ from the cube's `ERR` extension (or from the continuum scatter without one), with channels flagged `DO_NOT_USE` in the `DQ` extension dropped. Velocities come from the spectral axis and the rest wavelength: $v = v(x_c)$ and $\sigma_v = |v(x_c + \sigma_x) - v(x_c)|$. The integrated flux and its error are

$$F = A \sigma_v \sqrt{2\pi}, \quad \sigma_F^2 = 2\pi [(\sigma_v \sigma_A)^2 + (A \sigma_{\sigma_v})^2 + 2A \sigma_v \text{cov}(A, \sigma_v)] \quad (77)$$

in `BUNIT`·km/s. With `ERR` weighting the errors are absolute and inflated by $\sqrt{\chi_{\text{red}}^2}$ where $\chi_{\text{red}}^2 > 1$; without `ERR` they are scaled by $\sqrt{\chi^2/\text{dof}}$; every error includes the continuum uncertainty. Given a resolving power R , the instrumental width $\sigma_{\text{inst}} = c/(2\sqrt{2\ln 2} R)$ is removed, $\sigma_{\text{corr}} = \sqrt{\sigma_v^2 - \sigma_{\text{inst}}^2}$, and the spaxel is NaN with mask bit 16 when $\sigma_v \leq \sigma_{\text{inst}}$.

A two-component fit is tried where the single fit passes the S/N threshold and at least 8 channels are usable; the pair is kept only when the centres are at least one channel apart, each component is wider than the sigma lower bound and narrower than a quarter of the window, carries at least 10 % of the flux and has its centre inside the window, and, in `auto` mode, when

$$\frac{\text{BIC}_1 - \text{BIC}_2}{\max(1, \chi_{\text{red},1}^2)} \geq 10 \quad (78)$$

with both amplitudes at least 3σ . Dividing by the single fit's χ_{red}^2 keeps a misfitting single Gaussian from accepting every pair. The mask bits are: 1 fitted at or above the S/N threshold, 2 constant continuum, 4 DQ-dropped channels, 8 ERR-dropped channels, 16 unresolved, 32 not converged, 64 pair rejected or not attempted, 128 pair accepted. Velocities are in the frame named by SPECSYS; no barycentric correction is applied, and the notes say so.

3.28 Tabulated Spectral Axes (v0.6.7)

A spectral axis whose CTYPE ends in -TAB (FITS WCS Paper III, Greisen et al. 2006, §6) takes its coordinates from a binary-table column: `PSi_0` names the extension, `PSi_1` the coordinate column, `PSi_2` an optional index column, `PVi_1` the EXTVER (default 1) and `PVi_3` the coordinate-array axis, which must be 1. Only separable one-axis tables ($M = 1$, shape $[K]$ or $[1, K]$) are accepted. The intermediate coordinate of the 1-based pixel p is

$$\psi = \text{CRVAL}_i + \delta_i (p - \text{CRPIX}_i) \quad (79)$$

with the usual CD/CDELTA/PC step δ_i (each default recorded in the axis notes). Without an index column the 1-based table index is $v = \psi$; with a monotonic index column I it is found by inverse linear interpolation in I , accepted half a spacing beyond either end. The coordinate follows wcslib's `tabx2s`, including its linear extrapolation in the half-pixel margins: with K entries $C_0 \dots C_{K-1}$,

$$v \notin [0.5, K+0.5] \Rightarrow \text{NaN}; \quad p_1 = \lfloor v \rfloor, \delta = v - p_1, p_0 = p_1 - 1; \quad C(v) = C_{p_0} + \delta (C_{p_0+1} - C_{p_0}) \quad (80)$$

where $p_1 = 0$ shifts to $p_0 = 0$, $\delta - 1$ and $p_1 = K$ to $p_0 = K - 2$, $\delta + 1$. For $C = [1, 1.5, 2.25] \mu\text{m}$ this gives $\psi = 0.5 \rightarrow 0.75$, $1.5 \rightarrow 1.25$, $3.25 \rightarrow 2.4375$ and NaN below 0.5 or above 3.5. The unit is CUNIT_i, else the column's TUNIT (with a note), else the FITS default. The axis carries `tabulated: true` and, for code that needs one, a linear approximation (`crval` the first entry, `cdelta` the median step).

Every consumer with a file gets the tabulated axis: cube spectra, line measurement and line-fit maps, moments and collapse, and the x1d comparison. A PV diagram can only write a linear CDELTA2, so a PV from a -TAB axis is written when the table deviates from linear by at most 1 % of its step (with a HISTORY line giving the deviation) and refused otherwise. JWST MIRI MRS cubes carry a WCS-TABLE extension but a linear WAVE axis without PS3 cards; the app keeps using the linear axis there, which agrees with the table to one float32 ulp (at most $9.5 \times 10^{-7} \mu\text{m}$ on the local channel 4 cubes).

3.29 Reprojection Through the WCS (v0.6.7)

When the reference channel and another channel both have a celestial WCS, Compose Align reprojects the channel onto the reference pixel grid instead of resampling it by array size and shifting it. For each output pixel (X, Y) of the reference grid,

$$\begin{aligned}
(\alpha, \delta) &= W_{\text{ref}}(X, Y), & (x, y) &= W_{\text{ch}}^{-1}(\alpha, \delta), \\
O(X, Y) &= \begin{cases} \text{bicubic}(I_{\text{ch}}; x, y) & -0.5 \leq x \leq n_x - 0.5 \text{ and } -0.5 \leq y \leq n_y - 0.5 \\ \text{NaN} & \text{otherwise} \end{cases} \quad (81)
\end{aligned}$$

The mapping is evaluated exactly on a node grid of 8px in the reference frame and interpolated bilinearly between nodes (below 10^{-3} px for TAN-to-TAN mappings with a $2\times$ scale and 0.5° rotation). Pixel values are never rescaled: interpolation preserves surface brightness (MJy/sr) or counts per original pixel, and the per-pixel calibration cards of the output header are rescaled by the pixel-area ratio $A_{\text{ref}}/A_{\text{ch}} = |\det \mathbf{CD}_{\text{ref}}|/|\det \mathbf{CD}_{\text{ch}}|$. On the heavyTest pair, $\text{PIXAR_SR}(\text{F444W}) \times 0.246403 = 2.29183 \times 10^{-14}$ sr, the F200W pixel solid angle.

Coarser reference. When the reference pixel is coarser by a ratio $\rho \geq 1.5$, the channel is first box-reduced by $k = \max(\lfloor \rho \rfloor, 2)$ with its WCS adjusted ($\mathbf{CD}' = k \mathbf{CD}$, $\text{CRPIX}' = (\text{CRPIX} - 0.5)/k + 0.5$, SIP coefficients rescaled so block centres keep their sky positions), because point-sampled bicubic interpolation of undersampled stars does not conserve their flux.

Relative rotation is reported with a pinned sign: with $\mathbf{M} = \mathbf{CD}_{\text{ref}}^{-1} \mathbf{CD}_{\text{ch}}$, $\theta = \text{atan2}(M_{10} - M_{01}, M_{00} + M_{11})$, positive when the channel's pixel axes are rotated counter-clockwise relative to the reference ($+0.486^\circ$ for F444W on F200W).

Point-source residual. After reprojection, the remaining offset is measured on stars, not by phase correlation alone. Stars are detected at 5σ in up to 3×3 windows of at most 512px on both the reference and the reprojected channel; each of up to 30 reference stars per window is matched to the nearest reprojected star within 3px of its position plus a seed. There are two seeds, (0,0) (the WCS prior) and the phase-correlation offset; the seed with the most agreeing matches wins, ties going to (0,0). The residual is the median displacement of the matches that lie within 1px of the median, with their rms; fewer than 4 agreeing stars means no measurement. The residual is applied (the reprojection is shifted by it) only above 1px, and a residual above 5px is also warned. Without point sources the reprojection alone is stored and the note reads “residual not measured”.

Why phase correlation is only a seed

Above 512px the phase-correlation refinement runs on one central 512×512 crop. On the Pillars F200W/F444W pair that crop is extended emission that differs between the bands, and phase correlation reported an offset of $(-1.69, -1.31)$ px; applying it put the stars a median 2.20 px off. Measured on 219 matched stars (177 agreeing, rms 0.344 px) the residual is $(0.079, 0.045)$ px, below the 1px floor, so the reprojection is stored as is; an independent centroid check on the written file gives a median separation of 0.175 px.

3.30 Exact Rotations and Flips (v0.6.7)

Processing > Geometry rotates an image by 90, 180 or 270 degrees clockwise or flips it horizontally or vertically. For an image of H rows and W columns each operation is a signed permutation $\mathbf{M}$ with offset $\mathbf{t}_0$ on 0-based pixels:

$$\begin{aligned}
\text{rot90} : (x, y) &\mapsto (H-1-y, x), & \text{rot270} : (x, y) &\mapsto (y, W-1-x), \\
\text{rot180} : (x, y) &\mapsto (W-1-x, H-1-y), \\
\text{flip}_h : (x, y) &\mapsto (W-1-x, y), & \text{flip}_v : (x, y) &\mapsto (x, H-1-y)
\end{aligned} \quad (82)$$

In FITS 1-based pixels $\mathbf{p}'_f = \mathbf{M}\mathbf{p}_f + \mathbf{t}_f$ with $\mathbf{t}_f = \mathbf{t}_0 + \mathbf{1} - \mathbf{M}\mathbf{1}$. Because $\mathbf{M}^{-1} = \mathbf{M}^\top$, the world coordinate $\mathbf{CD}(\mathbf{p} - \text{CRPIX}) = \mathbf{CDM}^\top(\mathbf{p}' - \text{CRPIX}')$ is preserved exactly by

$$\boxed{\begin{aligned} \text{CRPIX}' &= \mathbf{M} \text{CRPIX} + \mathbf{t}_f, & \mathbf{CD}' &= \mathbf{CD} \mathbf{M}^\top, \\ \text{LTM}' &= \mathbf{M} \text{LTM}, & \text{LTV}' &= \mathbf{M} \text{LTV} + \mathbf{t}_f \end{aligned}} \quad (83)$$

PC/CDELTA/CROTA forms are converted to CD first. SIP polynomials are defined on $\mathbf{u} = \mathbf{p} - \text{CRPIX}$, which transforms as $\mathbf{u}' = \mathbf{M}\mathbf{u}$, so $(A', B') = \mathbf{M}(A \circ \mathbf{M}^\top, B \circ \mathbf{M}^\top)$ and likewise for AP, BP ; for signed permutations this is an exact relabelling and sign change of the coefficients, with the orders and DMAX values swapped on the transposing operations. Cards that describe the original pixel axes and that the app does not transform (ORIENTAT, PA_APER, PA_V3, VPARITY, ROLL_REF and every alternate-WCS card) are removed with a HISTORY line. On the F200W header all five operations reproduce the sky positions of the four corners, the centre and an off-centre pixel to 0.00 arcsec.

3.31 gWCS Evaluation and the Exact Inverse (v0.6.7)

A gWCS is a pipeline of frames joined by transforms; for JWST _cal and Roman L2 imaging it has three steps (detector $\rightarrow$ v2v3 $\rightarrow$ v2v3vacorr $\rightarrow$ world), and four for a tweakreg-corrected _crf, which adds v2v3corr before world. `core/astrometry/gwcs/` parses the ASDF tree into a node graph and evaluates it in f64, scalar and allocation-free. The supported tags are `compose`, `concatenate`, `shift`, `scale`, `polynomial` (1-D and 2-D with `domain/window`), `remap_axes`, `identity`, `affine`, `rotate2d`, `rotate_sequence_3d` (Cartesian and spherical), `spherical_cartesian`, `gnomonic` (both directions), `rotate3d`, the arithmetic operators `add`, `subtract`, `multiply`, `divide`, `power`, and `constant`; every other tag (`tabular`, `fix_inputs`, `jwst_pipeline/*`, `unknown`) is refused with its name and tree path. Semantics follow astropy and gwcs to rounding, including wcslib's special cases ($\phi = 0$ at $R = 0$ in `gnomonic`, NaN for $\theta \leq 0$ in `sky2pix`, the $+360^\circ$ wrap of a negative longitude in `rotate3d`). Stored inverses win at every nesting level; derived inverses exist for the invertible tags; only the first step's bounding box is applied, as gwcs does. ASDF writes gwcs tags in the verbatim form `!<tag:...>`, which `serde_yaml` 0.9 discards, so the parser rewrites them to the shorthand form before parsing; without that, `icrs` and `galactic` frames would be indistinguishable.

Exact inverse. The analytic backward chain (stored polynomial inverses, accurate to 0.000 45 px on NIRCcam and 0.15 px on FGS) only starts a Newton iteration on the forward chain. With $\mathbf{t}$ the unit vector of the target (α, δ) , the east and north vectors at the target $\mathbf{e} = (-\sin \alpha, \cos \alpha, 0)$ and $\mathbf{n} = (-\sin \delta \cos \alpha, -\sin \delta \sin \alpha, \cos \delta)$, and $\mathbf{u}(\mathbf{p})$ the unit vector of the forward evaluation,

$$\mathbf{r}(\mathbf{p}) = \begin{pmatrix} \mathbf{e} \cdot (\mathbf{t} - \mathbf{u}(\mathbf{p})) \\ \mathbf{n} \cdot (\mathbf{t} - \mathbf{u}(\mathbf{p})) \end{pmatrix}, \quad \mathbf{J}_{:,j} = \frac{1}{2h} \begin{pmatrix} \mathbf{e} \cdot (\mathbf{u}(\mathbf{p} + h\hat{\mathbf{j}}) - \mathbf{u}(\mathbf{p} - h\hat{\mathbf{j}})) \\ \mathbf{n} \cdot (\dots) \end{pmatrix}, \quad \mathbf{p} \leftarrow \mathbf{p} + \mathbf{J}^{-1}\mathbf{r} \quad (84)$$

with $h = 0.01$ px. The residual lives on the tangent plane of the sphere, so RA wrap and the poles need no special case. The iteration stops when the step is below 10^{-7} px (at most 10 steps) or diverges ($|\det \mathbf{J}| < 10^{-30}$ or a step above 10^4 px). A non-converged result is accepted only when the forward evaluation of the returned pixel is within 0.5 px of the target; otherwise the answer is NaN, never the analytic start, because far outside the field the stored inverses fold sky points back inside the bounding box. When the analytic start lies more than 8 px outside the bounding box, it is returned without Newton (dense off-footprint use such as reprojection). Measured (release build, 10^5 random points in the NIRCcam bounding box): forward 271 ns,

analytic inverse 274 ns, exact inverse $3.75\ \mu\text{s}$ per point, 2.0 iterations on average, maximum error 1.1×10^{-8} px.

Linear surrogate. Code that reads **CRPIX**/**CRVAL**/**CD** directly (region size conversions, the pixel-area fallback of photometric calibration, orientation) receives a surrogate in gWCS mode: **CRPIX** the bounding-box centre +1, **CRVAL** its forward evaluation, and **CD** the local Jacobian there ($h = 0.5$ px, the RA row multiplied by $\cos \delta$). Across a NIRC*am* field this differs from the full model by the $\leq 1\%$ distortion; Roman pixels are not square, and the surrogate reports $0.1103'' \times 0.1083''$.

3.32 TAN-SIP Fit of a gWCS (v0.6.7)

FITS cannot hold a gWCS, so derived outputs of a Roman image need a header approximation. `gwcs/sip_fit.rs` ports `gwcs WCS.to_fits_sip`. The reference pixel is the bounding-box centre rounded to 0.1 px (half to even, as Python's `round`); the forward model is sampled on an $n \times n$ grid over the bounding box ($n = 32$) and projected to the TAN plane at **CRVAL** with **LONPOLE** = 180. Polynomials without a constant term,

$$\xi(u, v) = \sum_{1 \leq i+j \leq d} a_{ij} u^i v^j, \quad \eta(u, v) = \sum_{1 \leq i+j \leq d} b_{ij} u^i v^j, \quad (u, v) = \mathbf{p} - \mathbf{CRPIX} \quad (85)$$

are fitted by LU-solved normal equations for $d = 1, 2, \dots, 6$ until the maximum residual is below 0.01 px (in units of the plate scale $\sqrt{|\det \partial(\xi, \eta)/\partial(u, v)|}$), with a check on a $2n$ grid. The linear terms form **CD** and the higher terms the SIP coefficients, $(A_{ij}, B_{ij}) = \mathbf{CD}^{-1}(a_{ij}, b_{ij})$ for $i + j \geq 2$. The inverse polynomials AP/BP (linear terms included) are fitted the same way to 0.01 px. The header receives **RA---TAN-SIP/DEC--TAN-SIP**, **CRPIX**, **CRVAL**, **CD**, the SIP coefficients in shortest round-trip exponent form, and the two residuals **SIPMXERR** and **SIPVERR**. On the Roman WFI01 F129 L2 file the fit selects degree 5 (degree 3 would leave 0.109 px, degree 4 0.074 px): **SIPMXERR** 3.48×10^{-7} px, **SIPVERR** 9.53×10^{-4} px, and the header SIP agrees with the gwcs truth to 3.5×10^{-5} mas over 122 points. The JWST pipeline writes its own TAN-SIP with the same targets (and **npoints** 12), which is why a JWST `_cal` header is kept rather than refitted.

4 World Coordinate Systems

One type, `WcsTransform` (`core/astrometry/wcs.rs`), answers every pixel-to-sky and sky-to-pixel question in the app. Since 0.6.7 it has two backends: **header**, the **wcs** crate's projection engine with SIP applied by AstroBurst's own code, and **gWCS**, the evaluator of Section 3.31. Every production consumer obtains it through one loader, `infra::wcs_source::load_wcs(path, header)`: the desktop WCS commands, photometry, time series, regions, cutouts, cubes, the catalog overlay and cross-match, observation geometry, SPCC, Compose Align and the server routes. Only two places build a header WCS directly, both on a header they construct themselves: the plate-solve probe and the pre-reduced channel of a coarser-reference alignment.

4.1 Header WCS: the Celestial Axis Pair (v0.6.7)

A header is accepted as a celestial WCS only when

- **CTYPE1** and **CTYPE2** are both present (a missing **CTYPE** is a linear axis in the FITS standard, so it is never read as **RA---TAN**);

- after stripping a -SIP suffix, both are 8-character ASCII types with longitude on axis 1 and the matching latitude on axis 2 (RA/DEC, GLON/GLAT, ELON/ELAT);
- both carry the same three-letter projection code.

Each failure has its own message (“CTYPE1/CTYPE2 missing: the header has no celestial axis types (FITS treats a missing CTYPE as a linear axis), so no sky WCS is read”, “... puts latitude on axis 1; only longitude-first celestial headers are supported”, “... are not a celestial longitude/latitude pair”, “... projection ... differ”). The message reaches the user as the tooltip of the amber “(no WCS)” note when the grid or compass is requested, and as the rejection of `get_wcs_info`. Before 0.6.7 a 2-D spectrum with CRPIX/CRVAL/CDELTA but no CTYPE was read as a TAN image at a pixel scale of 2700 arcsec, and its grid, compass, catalog cone and SPCC ran on invented coordinates. All local `i2d` and `s3d` products carry a valid pair and are unaffected.

4.2 SIP Fit Residual Cards (v0.6.7)

A JWST `_cal` imaging header carries a TAN-SIP fit of the gWCS with its residuals: `SIPMXERR` (maximum forward difference, in pixels) and `SIPIVERR` (maximum inverse difference). They are read when finite and non-negative, reported as `sip_max_err_px` and `sip_inv_err_px` by `get_wcs_info` and the server `wcs` route, and appended to the Info orientation line, for example `TAN - 0.031"/px x 0.031"/px - rot ... - E left - SIP (gWCS fit ≤0.010 px, inverse ≤0.012 px)`. Both are in pixels of the grid they were fitted on, so drizzle multiplies them by its output scale and resample divides them by the smaller of the two axis scale factors; a header whose WCS is replaced (Write WCS, a copy without WCS) loses them, so a stale residual never sits next to a new solution.

4.3 gWCS Sources and Policy (v0.6.7)

`wcs_source` looks for a gWCS once per file and caches the outcome by path, size and modification time (16 entries; a hit, a “no gWCS” and a refusal are all cached). The tree is classified before it is parsed, so only a two-pixel-axis imaging gWCS ending in an ICRS (or FK5) celestial frame is ever evaluated:

Product	Where the gWCS is	WCS used
JWST <code>_cal</code> , <code>_crf</code> imaging	ASDF BINTABLE extension of the FITS file (one byte column holding a complete ASDF file)	gWCS
JWST <code>_i2d</code>	same, but a <code>fitswcs_imaging</code> transform equivalent to the header TAN	header
JWST <code>_s3d</code> , <code>_s2d</code> , WFSS, NIRSpec IFU <code>_cal</code>	3-D or spectral world frames	header
JWST <code>_rate</code>	no <code>meta.wcs</code> before <code>assign_wcs</code>	header
Roman L2 <code>_cal.asdf</code>	<code>roman.meta.wcs</code>	gWCS
Roman <code>_wcs.asdf</code> sidecar	<code>roman.wcs_12</code> , <code>roman.wcs_11</code>	parsed in tests only
Any imaging gWCS with an unsupported transform or frame	—	header, with the refusal shown

Table 3: Which WCS a file gets. A refusal is never silent: the Info line ends with “gWCS not used: <reason>” (for example “gWCS transform ‘tabular’ at steps[0].transform”), and `get_wcs_info` returns it as `gwcs_refusal`.

JWST `_cal`. When the ASDF extension holds a supported imaging gWCS, it is the WCS; the

header TAN-SIP stays in the header as the fallback and is compared with the gWCS on a 9×9 grid over the bounding box. The residual cards are taken from `meta.wcsinfo` (the pipeline's own fit report) when the ASDF carries them. The Info line then reads, for example, `gwcs (3 steps: detector→v2v3→v2v3vacorr→world) - 0.031"/px x 0.031"/px - rot ... - header SIP within 0.009 px (0.27 mas)`.

Roman. An ASDF image is served in gWCS mode only; its synthesised FITS header carries the fitted TAN-SIP cards of Section 3.32 so header-only code and derived FITS have a WCS. Before 0.6.7 the reader folded a gWCS into TAN only when its chain was shift, scale, affine, gnomonic and rotation, and refused the Roman distortion chain at its first `remap_axes`, so Roman L2 images opened without sky coordinates.

Bounding box. The user-facing readouts, `pixel_to_world_cmd` (hover, status strip) and the server `pix2sky`, apply the gWCS bounding box and return `null` outside $[-0.5, N - 0.5]$ as `gwcs` does. Every internal consumer extrapolates like a header WCS, because reprojection nodes, the region position-angle probe, the ruler and PV start points can legitimately sit just outside the image.

Reported keys. `get_wcs_info` and the server `wcs` route add `wcs_kind` ("header" or "gwcs"), `gwcs_steps`, `gwcs_frames`, `gwcs_source` ("ASDF HDU 8", "roman.meta.wcs"), `gwcs_vs_header_sip_max_mas`, `gwcs_vs_header_sip_max_px` and `gwcs_refusal`; in gWCS mode `projection` is "gwcs" and the linear fields are the surrogate of Section 3.31.

Data	Result
NIRCam <code>nrca1_cal</code> (jw02739, cal 3.0.0)	gWCS from ASDF HDU 8; test pixels within 10^{-9} deg of <code>gwcs</code> 1.0.3; header TAN-SIP within 0.266 mas (0.009 px) of the gWCS on the 9×9 grid; <code>SIPMXERR</code> 0.00874 px from <code>meta.wcsinfo</code>
MIRI imager <code>_cal</code> (cal 3.0.0)	exact inverse of the world position of pixel (0,0) returns it to 5×10^{-11} px; the header AP/BP inverse is off by (0.018, 0.031) px
Roman WFI01 F129 L2	gWCS from <code>roman.meta.wcs</code> ; test pixels within 10^{-9} deg; surrogate scale $0.1103'' \times 0.1083''$; grid of $7 + 7$ lines; fitted header of degree 5
28 JWST i2d products	header mode, corners identical to the header path to 10^{-12}
<code>gwcs</code> 1.0.3 truth sources	22 evaluated; the spectroscopic ones refused

Table 4: Real-data validation of the gWCS integration (#[ignore] tests reading local data through environment variables).

4.4 Derived FITS Outputs

Derived files inherit a header WCS through the existing card-copy rules: a cutout shifts `CRPIX` and records LTV/LTM, crop and resample rescale the SIP coefficients, drizzle and resample rescale the residual cards, and Geometry transforms the WCS exactly (Section 3.30). For a Roman image these are the fitted TAN-SIP cards, so a 400×400 cutout with only `CRPIX` shifted agrees with the parent gWCS to about 6×10^{-6} mas. For a JWST image the pipeline's own TAN-SIP is kept, and a cutout adds `SIPMXERR`/`SIPIVERR` from `meta.wcsinfo` when the header lacks them. No per-output refit is done.

4.5 Plate Solving and Write WCS

Plate solving uploads the image to nova.astrometry.net and needs a free API key in Settings; there is no local solver (`solve_offline_placeholder` always fails with a message). The scale range is hinted at 0.8 to 1.25 times the header pixel scale and the centre from the header WCS when one exists; the centre and radius fields accept a dot decimal only. Large images are downsampled for the upload and the solution is rescaled to the full grid. A reply without a usable RA, Dec or positive pixel scale is an error rather than a solve at (0,0).

Write WCS (0.6.5.0) writes `<name>_wcs.fits` with the astrometry.net TAN-SIP solution, replacing every old celestial WCS card (CD, PC, CDELT, CROTA, SIP, PV and the residual cards) and setting `ABPROC='platesolved'`, then shows it as “Plate-solved WCS” so the grid, compass, Targets, photometry and the Gaia catalog use it. Since 0.6.7 both the solve and the write run on the image being measured, the processed result on screen, not the original file: before, stars were detected on a stack while the first raw frame was uploaded, and Write WCS replaced the stack on screen with that frame. The Plate Solve header shows the same source badge as the other measurement panels, the output is `{processed_stem}_wcs.fits`, and a processing result published while a solve is running cancels that solve.

Retries (0.6.7). Every GET of a solve (submission status, job status, calibration, annotations, WCS file) is attempted up to three times with 1 s and 2 s backoff when the failure is a transient transport error or an HTTP 5xx; the login and the upload are not retried. Idle pooled connections are dropped after 4 s, under the server’s keep-alive timeout of 5 s, and error messages carry the full cause chain. The defect this closes: a job that astrometry.net had solved was reported as “error sending request for url (.../calibration)” after one dropped connection. In the release acceptance run a real solve of a background-corrected NIRCcam F200W image followed by Write WCS produced `..._bg_corrected_wcs.fits` with its centre at $(274.729885^\circ, -13.851857^\circ)$ and $0.03122''/\text{px}$.

5 Measurement and Science Analysis

5.1 The Measurement Contract

A measurement is taken on the image on screen: the processed result when there is one, otherwise the file. Each measurement panel carries a badge naming its source (“Values from Background”, “Plate-solved WCS”, “selected file” while the colour composite is shown). Version 0.6.7 closed the three places that broke the rule:

- **Info panel sky readout.** It reads the analysis target and hides itself when the target has no celestial WCS (a PV diagram, whose first axis is `OFFSET`) or when a PNG-only result sits on another grid than the file. Before, hovering a PV diagram printed the RA/Dec of the cube spaxel with the same pixel numbers.
- **Plate Solve and Write WCS** run on the measured image (Section 4.5).
- **Stale numbers after a rewrite.** Statistics, region statistics, region profiles and the line sky readout re-run on a key made of the path and a render version (`path@version`), so re-running Background with another degree, which rewrites the same `_bg_corrected.fits`, refreshes them without a click.

A click on a result whose grid differs from the cube’s spatial grid (a PV diagram) no longer extracts a spectrum; before, it plotted the spectrum of the spaxel with the same numbers, or failed with a row index beyond the cube. Ramps keep their own pixel mapping.

5.2 Regions and Exclusions

Circles, ellipses, boxes, annuli and polygons have an include/exclude toggle (0.6.5.0), the DS9 - prefix, kept by `.reg` import and export. The pixels inside every exclude region are left out of the other regions' statistics and background annuli and of radial profiles, line cuts and surface-brightness profiles, merged with the DQ mask when DQ exclusion is on; the `n` cell reads (`-N excl`) and counts both causes, and an exclude region's own row is measured on its own pixels. `region_stats_cmd`, `radial_profile_cmd`, `line_cut_cmd` and `sb_profile_cmd` take at most 512 exclude shapes; an invalid shape (zero radius, polygon with fewer than three points) is skipped and named in `notes`. When the header carries a flux calibration and a WCS, every region also reports flux in Jy, AB magnitude, surface brightness, area, centre RA/Dec and sky position angle (0.6.2).

5.3 Photometry

Aperture photometry (Section 3.20) runs on a click with the Crosshair tool, on a list in the Photometry table (detected stars, Point regions or pasted positions, with apertures drawn on the image), or across frames as a time series with drift tracking and a differential light curve. The gain field is prefilled from the model of Section 3.25 with a caption naming the source, and click and batch responses report `gain_used`.

The optional Gaia DR3 match of click photometry (opt-in, remembered) queries one field cone with $G < 17$ through the catalog cache and matches locally within 5 arcsec. When that cone comes back at its row cap, so that stars brighter than $G = 17$ may have been cut, the match falls back to a 0.01° cone around the star, also cached. Before 0.6.7 every click was one VizieR request and nothing was cached. A failed query, an empty cone or a missing WCS is a warning, never a missing box.

5.4 Catalogs and the Disk Cache (v0.6.7)

Gaia DR3 cone searches go through three tiers: the in-process LRU (16 cones), a disk cache, and VizieR. A cone is keyed by its centre and radius quantised to 10^{-4} deg, the magnitude limit to 0.01 and the row limit, and stored as one JSON file per key (`gaia-dr3_{ra}_{dec}_{r}_{mag}_{rows}.json`, written to a temporary file and renamed). The directory is `dirs::cache_dir()/astroburst/catalog-cache` (`%LOCALAPPDATA%` on Windows, `~/Library/Caches` on macOS, `~/.cache` on Linux), capped at 64 MiB with the oldest files evicted first. Gaia DR3 is a fixed release, so entries have no expiry and the network is used only on a miss; an empty result is never written, so a bad response cannot poison the cache. Cone search and cross-match report `catalog_source` (`network`, `memory` or `disk`).

Two environment variables are read once at start, by the desktop app and the server: `ASTROBURST_CATALOG_CACHE_DIR` moves the cache, and `ASTROBURST_VIZIER_URL` replaces the VizieR endpoint (a mirror, or an unreachable address to test offline behaviour). CSV exports write numbers exactly as JavaScript's `String(number)` does (exponent form below 10^{-6} and from 10^{21}), so a file saved from Rust and a clipboard copy from the frontend agree byte for byte; `serde_json` is built with `float_roundtrip` so the IPC body keeps every digit.

5.5 Filters

The filter-wavelength table exists in TypeScript (`src/utils/filterWavelengths.ts`) and, since 0.6.7, in Rust (`core/metadata/filter_wavelengths.rs`), with a vitest that parses the Rust table

and requires both to hold the same 74 codes. Values are the nominal code wavelength (F444W $\rightarrow$ 4440 nm, F158M $\rightarrow$ 1580 nm), not pivots. The effective filter of a header follows one rule on both sides: when PUPIL holds a known code (a NIRCam pupil-wheel filter such as F470N) it wins; otherwise FILTER, FILTER1, FILTER2, FILTNAM1, FILTNAM2 and finally PUPIL are tried, skipping blanks and one- or two-digit wheel positions. Time-series photometry uses it to skip a frame taken through another filter, now compared case-insensitively.

5.6 Spectra

- **Spectral axes.** Linear axes as in 0.6.0 (Section 3.21), plus tabulated -TAB axes (Section 3.28); -LOG and other non-linear codes are refused with a reason.
- **Binary tables.** The BINTABLE reader (0.6.5.0) reads scalar columns and, since 0.6.7, fixed-repeat array columns (rE, rD, rB, rI, rJ, rK) as flattened row-major vectors, shaped by TDIM (whose product must equal the repeat) with TNULL/TSCAL/TZERO applied per element. Variable-length P/Q columns are refused with a message. A table is located by EXTNAME and EXTVER from the raw headers, so an unrelated table with a layout the parser rejects (a JWST ASDF or HDRTAB extension) does not hide it.
- **Pipeline x1d.** “Compare with pipeline x1d” (0.6.5.0) reads the EXTRACT1D table next to a cube, resamples it onto the cube’s channels in vacuum μm without crossing DQ-flagged rows, and plots FLUX (Jy) or SURF_BRIGHT beside the region spectrum; a header that declares more data than the file holds is refused.
- **Wavelength readout.** On a frame with a same-size WAVELENGTH extension (NIRSpec_cal), the status strip and the pixel table show the wavelength at the cursor (0.6.5.0).
- **Line list and measurement.** 27 rest-frame lines (0.6.3, 0.6.6) drawn at a typed redshift or systemic velocity; line measurement over a brushed range with an optional Gaussian fit (0.6.2); spectrum export and comparison with provenance lines (0.6.3).

5.7 Cubes

Region spectra, range collapse and moment maps are unchanged from 0.6.0 (Section 3.22). Line-fit maps (0.6.6, Section 3.27) fit a brushed line in every spaxel in a fraction of a second on a NIRSpec IFU cube and write one 2-D FITS per plane (flux, velocity, sigma_obs, sigma_corr, the errors, chi2_red, snr, mask, ncomp and, for accepted pairs, twelve c1_*/c2_* planes) with the celestial WCS and LF* provenance cards. A click on a displayed plane refits that spaxel and draws a mini plot of the fit from the arrays the backend returns. Velocity planes open in an inverted RdBu map symmetric about zero, a temporary display that is never saved. Position-velocity diagrams (0.6.3) sample a cube along a line region without loading it whole, with offsets in arcsec from the CD matrix, and are written as a 2-D FITS with an OFFSET axis and the native spectral axis.

5.8 Ramps (v0.6.6)

A 4-D FITS image with NGROUPS/NINTS cards, or a DATAMODL of Level1bModel/RampModel, opens as a ramp (JWST_uncal, Roman L1 resultant stacks) with group and integration navigation, the readout identity (including the NIRSpec IRS2 layout and the group time) and a per-pixel ramp plot with saturated, unused and jump-flagged groups. “Run quick slope” turns one integration into <stem>_qslope.fits with SCI (DN/s), NGOOD, DQ and NOISE extensions, offline and without reference files: the IRS2 reference output and interleaved reference rows are stripped, a per-amplifier reference offset is subtracted, saturation is flagged by threshold and jumps crudely, and the slope is an ordinary least-squares or median fit. It is a decision aid, not a rate: there is

no superbias, linearity, dark or weighted fit, and the header carries `ABPROC = qslope` and never `CAL_VER`, `S_RAMP` or `DATAMODL`. “Compare with rate” reads the official `_rate.fits` and reports the alignment check, per-bin and per-amplifier differences, `JUMP_DET` and `SATURATED` recall and precision, and a verdict against the published tolerances. On 8 NIRSpec IRS2 exposures of program 1266 the faint additive offset is within 0.011 DN/s over the full frame and 0.021 DN/s per amplifier, and the quick slope differs from the official one by +1.2 to +2.7 % at 1–30 DN/s (`docs/quick-slope.md`).

5.9 Time Series, Observation Geometry and the Log

Observation geometry (0.6.3) reports `BJD_TDB` and `HJD.UTC` of mid-exposure (leap-second table, TT and TDB, light time within 1 s of astropy), local sidereal time, hour angle, altitude and azimuth, Kastan-Young airmass, parallactic angle, and the Sun and Moon, for the loaded file and for every frame of a time series, whose light curve uses `BJD_TDB` when every frame has one. The measurement log (0.6.3, the Log tool since 0.6.6.3) records every photometry click, batch run, line measurement, line-fit run, time series, cross-match, spectrum export and PV run with its UTC timestamp, measured image, DQ handling, parameters and values; it is kept for the session and exports as CSV.

6 Calibration, Stacking and Compose

6.1 Frame Cards and Consistency Checks (v0.6.7)

Before any master is built, the headers of every bias, dark, flat, flat-dark and light frame are read once (merged primary and science header, so JWST MEF values are seen): exposure (Section 3.14), sensor temperature (`CCD-TEMP`, else `SET-TEMP`), effective filter, `IMAGETYP`, `GAIN`, `OFFSET`, `XBINNING`/`YBINNING` and the Bayer cards. The checks run identically in the Calibration tab, the desktop Pipeline and the server’s `pipeline/run`; a refusal fails before any pixel is loaded (400 on the server), and warnings travel in the response.

Rule	Outcome
Dimensions, binning, <code>GAIN</code> or <code>OFFSET</code> differ inside a master set	refuse
Dark exposures spread by more than 1 % of their median	refuse
Flats taken through different filters	refuse
A frame classified Light or Flat in the Bias, Dark or Flat-dark row	refuse
Lights binned differently from the masters	refuse
Flat exposures spread by more than 20 %	warn
Bias exposure above 0.5 s	warn
Flat-dark median exposure more than 5 % from the flats’	warn
Light exposure differs from the dark’s by more than 1 % with no bias (dark not scaled)	warn
Lights’ filter differs from the flats’	warn
Lights’ <code>GAIN</code> or <code>OFFSET</code> differs from the darks’ or the bias’	warn
A flat-dark frame in the Dark row; a light frame in the Flat row (sky flats)	warn
Flats calibrated by an unscaled light dark of another exposure (no bias, no flat-darks)	warn, quantified
Some darks with an exposure card and others without	warn, files listed

Table 5: Consistency rules. A missing card never refuses; an absent or unrecognised `IMAGETYP` is accepted silently, and the Flat-dark row accepts frames typed Dark or Bias, as acquisition programs often label them.

Flat-darks. A fourth master. When flat-dark frames are given, flats are calibrated as $F_i - M_{\text{flatdark}}$. Without them, a dark without bias is still subtracted from the flats unscaled (which removes the bias pedestal and over-subtracts the thermal signal by the exposure difference) and the warning quantifies it; the run is not refused, since that is the usual cooled-CMOS setup.

Temperature groups. Darks are sorted by temperature and split only at gaps larger than 3°C , so a set-point library at -20 and -10°C becomes two groups while a continuously drifting uncooled sensor stays one; a group of fewer than 5 frames is merged into its nearest neighbour. Each light takes the group whose mean temperature is nearest to its own. A group spanning more than 2°C , or a light more than 2°C from its group, is warned (dark current roughly doubles every 7°C , so 2°C is about 22 %); a run is never refused for temperature. Lights without a temperature use the largest group; when any dark lacks a temperature, all darks form a single group. With a $-20/-10^\circ\text{C}$ library and lights at -10°C a single averaged master under-corrected the dark current by 31 %.

Cosmetic correction on Bayer data. `CosmeticConfig.cfa` is optional: when absent, the CFA lattice is detected from `BAYERPAT/COLORTYP`, so a hot red site is replaced from red neighbours, not from green and blue ones; the panel shows the detected pattern and the response says which setting was applied. When the MAD of the residuals is zero, the robust scale now falls back to the 90th percentile of $|v - \text{median}|$ divided by 1.6449, and to the mean absolute deviation only when 90 % of the pixels sit exactly at the median, because a few hot pixels inflated the old fallback until a warm pixel at 50 quantisation levels was never flagged.

6.2 Stack Outputs and Combination (v0.6.7)

Stack-tab results, Pipeline channel masters and drizzle outputs carry their provenance in the header:

Card	Content
NCOMBINE	Frames included (after alignment exclusion); for inputs that are themselves AstroBurst stacks (NCOMBINE with ABCOMB) the sum of their NCOMBINE
EXPTIME	The per-frame exposure when all agree within 1 %, else the median with a HISTORY note
TOTEXP	Sum of the known exposures (of the inputs' TOTEXP for stacks); a HISTORY note when some are unknown; absent when none is known
ABCOMB	mean, median, min or max (drizzle writes mean)
ABREJECT, ABNORM	Rejection and normalization names (ABNORM omitted for drizzle)
ABDRZSCL	Drizzle output scale, 1–4
HISTORY	AstroBurst stack: N frames, <rejection>, <combine>, <normalization> or AstroBurst drizzle: N frames, scale S, pixfrac P, <kernel>

Table 6: Stack provenance cards. NCOMBINE, ABCOMB and ABDRZSCL are kept by linear outputs (background, crop, align) and dropped by rescaled and display-referred ones, since the gain model reads them; stale cards inherited from a reference frame are removed. The RGB drizzle FITS does not carry these cards yet.

The combination itself changed in three places: noise weights are divided by k_f^2 (Section 3.18); frames of different sizes are refused when alignment is off (before, they were cropped from the top-left corner and stacked misregistered) and listed in a warning when alignment is on; and the Stack panel disables its frame rows while a stack runs and shows the effective weights the backend used. One-shot-colour lights (BAYERPAT or COLORTYP) are refused before alignment or drizzle since 0.6.5.0, with the debayer route named.

6.3 Batch Pipeline

The Stack-tab Pipeline calibrates, registers and stacks each channel, writes one master FITS and one auto-STF PNG per channel plus the RGB quick look to the output folder (0.6.5.0), and leaves out the frames rejected in the Subframe Selector. Each frame is registered against the channel’s first frame by phase correlation, then by the affine matcher. Since 0.6.7 a frame that neither method registers is excluded from the stack. A warning in the Stack tab’s wording, shown in the Pipeline panel, identifies it by position, for example “Channel ‘L’: frame 3 of 3 was left out of the stack: it could not be aligned to frame 1 (identity, confidence 0.00)”. The master’s NCOMBINE and TOTEXP count only the stacked frames, and when fewer than two frames of a channel remain the run fails with an error naming the channel. Before, the pipeline stacked such a frame as it came out of the failed alignment. The Pipeline takes no method name; **stack**, **align_channels_cmd** and **export_aligned_channels_cmd** now reject an unknown one instead of running phase correlation.

6.4 Compose Wizard

The ten steps are unchanged; what they do changed as follows.

Stack. A stack that finishes after a Reset, or after its bin’s files or exclusions changed, is discarded with “Inputs changed while stacking; result discarded”, and Reset cancels a running stack.

Align. When the channels carry a celestial WCS they are reprojected through it (Section 3.29). The reference is the channel with the finest WCS pixel scale, channels within 0.5 % of the finest counting as tied and the first filled bin among them winning; a REF selector overrides it, and the response names the rule (**finest_wcs**, **selected** or **first**). Each channel’s note reads, for example, “reprojected through WCS (scale x2.0145, rotation +0.49°); residual Δx 0.0 px Δy +0.1 px”, or “... pre-reduced x2” when the reference is coarser; a gWCS channel is reprojected through its gWCS. A channel whose footprint does not overlap the reference is left empty with a warning. Without a WCS on either side the previous path (resample by array size, then registration) is used and the result says so. On the Pillars F200W/F444W pair the old array-size mapping left the F444W corners 95 to 149 px from their WCS positions; after reprojection the residual measured on stars is below 0.1 px. In the app the F444W channel is reprojected onto the 14344×8589 F200W grid in about 2 s.

Run tokens. Wizard cache keys carry a per-run token (`__wizard_ch_t<token>_<bin>_<stage>`); Crop and Background use the token of the Align run they continue, a run removes only keys of other tokens, and a run that finishes after a newer one removes its own keys. Before, an older Align finishing last overwrote the newer run’s pixels under shared keys.

Crop opens for a single filled channel (its stacked or raw file), not only after a two-channel Align.

Blend applies Match levels by median and MAD and decides the preview STF link on the composite output (Section 3.26).

Colour. SPCC is offered for broadband visible inputs and evaluates the blackbody model at each channel’s filter wavelength (Section 3.16); the option reads “SPCC (blackbody approximation)”, or “SPCC (broadband RGB only)” and disabled, with the filter named, when an input is narrowband, medium-band or outside 380–830 nm.

Stretch. Star removal no longer marks the Stretch step done; it invalidates Stretch and the later steps, keeping the starless channel.

Export. Without a blended composite, exporting two or more channels uses the fixed mapping

$R \leftarrow \{r, \text{sii}, \text{ha}\}$, $G \leftarrow \{g, \text{ha}, \text{oiii}\}$, $B \leftarrow \{b, \text{oiii}, \text{sii}\}$ (first unused bin wins, so $\{\text{H}\alpha, \text{OIII}, \text{SII}\}$ gives $R = \text{SII}$, $G = \text{H}\alpha$, $B = \text{OIII}$). A banner now shows that mapping, and the FITS gains an ASCII HISTORY line such as Channel mapping: `R=SII G=HA B=OIII (fixed, Blend weights not applied)`.

7 Processing Tools

The Processing tab holds Debayer, Background (polynomial and DBE), Denoise, PSF, Deconvolution, Stretch (arcsinh, GHS), Masked Stretch, LHE, HDRMT, Curves, PixelMath and Geometry. The chain stages are listed in Section 2.3; while the Compose composite is on screen, the steps that support it run on its three planes with the composite’s own chain (0.6.4). The algorithms of 0.6.0 are described in Section 3; this section covers what 0.6.7 added.

7.1 Curves and Levels on a File

The Curves panel applies levels (black point 0–0.5, gamma 0.2–5 on a logarithmic slider, white point 0.5–1) and one tone curve, edited with the same curve editor as Compose > Adjust, to a mono FITS or to each plane of an RGB FITS (`apply_tone_cmd`). The output is display-referred, so a linear input is first brought to display space with its own auto-STF, the mapping the viewer shows (one linked STF for the three planes of an RGB FITS, so the colour balance is kept), and a HISTORY line records the STF parameters; an input already marked `ABDISP=T` is taken as `[0, 1]`. Min/max normalisation was rejected because on linear data the sky sits near 0.7% of the range (WFPC2 502nm: median 7×10^{-3} after min/max scaling), where any black point above 0.01 clips the whole image. The result is written with `ABPROC='tone'` and `ABDISP=T`, joins the chain after LHE/HDRMT and before PixelMath, and is refused when both levels and curve are identity (“Nothing to apply: the levels and the curve are both identity.”) or the input is a cube. On the composite the panel points to Compose > Adjust.

7.2 Background and De-Banding on a File

The Background panel adds four modes to Subtract and Divide: de-band with an auto-detected axis, rows, columns or both, using the polynomial model’s sigma clip and 1–5 iterations; the “model” preview then shows the removed band pattern, the header gains `ABDEBAND` and the result reports the axis. De-banding was available only in the Compose Background step before. A guard refuses dispersed exposures, whose rows and columns are spectra: `EXP_TYPE` starting with `NRS_`, `MIR_MRS`, `MIR_LRS`, `NIS_SOSS`, `NIS_WFSS`, or a NIRCам grism mode (“De-banding is for imaging frames; this is a dispersed exposure ... so a band model would subtract signal.”). The modes are disabled under the Spline model and on the composite, whose background command has no de-band branch.

7.3 Geometry: Rotate and Flip

Five buttons rotate the displayed image by 90, 180 or 270 degrees clockwise or flip it horizontally or vertically (`transform_geometry_cmd`), with the exact WCS update of Section 3.30. Arbitrary angles are not offered. The input is the image on screen; the output is a new file, `<stem>_<op>.fits` with `ABPROC='<op>'`, added to the Files list, because a rotation inside the fixed-stage chain would either be dropped by a later re-run or leave the next step working on the unrotated original. RGB FITS planes are transformed together; cubes are refused; only the

science plane is written, and a `HISTORY` line says so when the source has ERR or DQ companions. Per-pixel calibration cards are unchanged, since the pixel area is preserved.

8 Command Module Reference

The backend registers **140 Tauri commands**, all of them listed in the `tauri::generate_handler!` macro in `src-tauri/src/lib.rs`; the table below groups them by their `cmd::` module path. Release 0.6.0 registered 107.

The command name strings appear nowhere in the React components. They live only in the service layer, `src/services/*.ts`, where each exported function passes the literal name to `typedInvoke` (`src/infrastructure/tauri/client.ts`). Arguments cross the boundary in camelCase and arrive in Rust as snake_case — `computeStatistics` sends `excludeDq` and `compute_statistics_cmd` receives `exclude_dq` — so a rename on either side is a runtime failure, not a compile error.

9 FITS and ASDF I/O

FITS via `memmap2` with BSCALE/BZERO: $v_{\text{phys}} = \text{BSCALE} \times v_{\text{stored}} + \text{BZERO}$. Multi-BITPIX export: 16 (auto BZERO/BSCALE), -32 (float32), -64 (float64).

ASDF: a native Rust reader. YAML tree $\rightarrow$ `NdArrayMeta` $\rightarrow$ binary block (zlib/bzip2/lz4) $\rightarrow$ `Array2`, with Roman data-model traversal and auto-dispatch by extension. The reader was rewritten in 0.5.8 around `memmap2` with lazily decoded blocks and validated against the ASDF Standard. Since 0.6.7 verbatim YAML tags (`!<tag:stsci.edu:gwcs/...>`) are kept, the gWCS of an image is evaluated (Section 4), and a Roman image’s synthesised header carries a fitted TAN-SIP WCS.

9.1 Image References (v0.6.0)

Every path-based command accepts a reference rather than a bare path: `<path>#hdu=<n>` selects any FITS HDU and `<path>#array=<key>` any ASDF array, including dotted keys such as `roman.dq`. Parsing lives in `types/image_ref.rs`; a single loader in `infra/image_source.rs` resolves references for both the desktop commands and the headless server. The reference, not the path, is the cache key, and output filenames derive their stem from it, so `jw01234_cal.fits#hdu=3` writes `jw01234_cal_hdu3.png`. This is what makes ERR, DQ and variance planes first-class images that can be displayed and measured like any other.

9.2 Lossless Integer Planes (v0.6.0)

A DQ plane is a bit field, so rounding it through `f32` destroys individual flags. BITPIX 8/16/32 HDUs and `int8–int32`, `uint8–uint32` and `bool8` ASDF arrays are therefore decoded a second time into an `IntPlane` that sits beside the float view in the cache and counts against the same budget. The BZERO offsets 32768 and 2147483648 that FITS uses to store unsigned data in signed types are applied in integer arithmetic, never through a float intermediate. Display and processing stay in `f32/f64`; only flag decoding reads the integer plane.

Module	Commands	Area
cmd::io	6	FITS/ASDF ingestion, linear mono and RGB preview buffers, RGB file as composite, cube release
cmd::export	5	FITS and PNG export, mono and RGB, RICE-compressed MEF
cmd::metadata	5	Headers, HDU and ASDF array listings, narrowband filter detection
cmd::analysis	9	Histograms, FFT power spectrum, star detection (mono and composite), click and batch photometry, gain model, time-series photometry, subframe metrics
cmd::regions	6	Region statistics, radial and surface-brightness profiles, line cuts, DS9 .reg import and export
cmd::visualization	9	STF render, scale limits, colormap LUT, pixel probe, pixel table, DQ flag table and mask preview, tile pyramids (mono and RGB)
cmd::stacking	5	Calibration, integration, drizzle (mono and RGB), batch pipeline
cmd::compose	20	Channel blend, level measurement, alignment and its overlay preview, crop and border detection, LRGB, white balance, SCNR, restretch, composite cache, aligned-channel export, the composite processing chain
cmd::processing	23	Resample, debayer (single and batch), deconvolution, background (polynomial with de-band, batch, DBE), wavelets, arc-sinh/GHS/masked stretch, star removal, tone on files and on the composite, geometry, LHE, HDRMT, composite variants
cmd::pixelmath	2	Expression evaluation and expression validation
cmd::cosmetic	2	Cosmetic correction, single frame and batch
cmd::statistics	3	Exact statistics (mono and composite) and batch noise evaluation
cmd::cube	6	Cube info, frames, pixel and region spectra, range collapse, moment maps
cmd::line_measure	1	Spectral line measurement
cmd::linefit	2	Line-fit maps and single-spaxel refit
cmd::ramp	7	Ramp info, pixel series, tables, frames, pixel fit, quick slope, comparison with <code>_rate</code>
cmd::spectral	3	Spectral axis (tabulated included), radial-velocity correction, <code>x1d</code> reader
cmd::pv	1	Position-velocity diagram
cmd::cutout	1	Science cutout export (SCI, ERR, DQ)
cmd::catalog	3	Gaia DR3 cone search, cross-match, CSV export
cmd::astrometry	8	Plate solving, Write WCS, WCS info, pixel-to-world, world-to-pixel, sky separation, pointing overlap, grid lines
cmd::geometry	1	Observation geometry (time scales, airmass, Sun and Moon)
cmd::contours	1	Contour lines
cmd::psf	1	PSF estimation
cmd::spcc	1	SPCC (blackbody colour calibration)
cmd::config	4	Configuration read and write, API key storage
cmd::synth	2	Synthetic frame and synthetic stack generation
cmd::output	3	Output directory info, cleanup, progress cancellation
Total	140	

Table 7: The 140 commands registered in `generate_handler!`, grouped by `cmd::` module. Counts read off the macro block in `src-tauri/src/lib.rs`; the four added in 0.6.7 are `photometry_gain_model_cmd`, `measure_channel_levels_cmd`, `apply_tone_cmd` and `transform_geometry_cmd`.

9.3 Multi-Extension Output (v0.6.0)

`write_mef_images` writes plain multi-HDU IMAGE files from `f32`, `i32` and `u32` arrays. The science cutout path uses it to emit SCI, ERR and DQ in one file with CRPIX shifted and LTV1/LTV2/LTM1_1/LTM2_2 recorded, so the cutout remains registerable against its parent. `export_fits` can additionally write RICE_1-compressed tiles with a quantization level, for BITPIX 16 or -32 only. Export with “Apply current STF stretch” (0.6.5.0) writes a display-referred header: `ABPROC='stf_export'`, `ABDISP=T`, no BUNIT, photometric or saturation cards, so region fluxes and photometry treat the file as uncalibrated.

9.4 Binary Tables (v0.6.5, v0.6.7)

Binary tables are not opened as images. `infra/fits/table.rs` reads columns by name for the consumers that need them: the JWST EXTRACT1D table of an x1d file, the WCS-TABLE of a -TAB spectral axis, and the ramp tables (GROUP, INT_TIMES). Fixed-repeat array columns are read since 0.6.7 (Section 5); the reader refuses a table whose header declares more data than the file holds, a TFIELDS outside 0–999 and a duplicated column name.

9.5 The JWST ASDF Extension (v0.6.7)

A JWST _cal FITS file carries an ASDF BINTABLE extension (HDU 8 in the local files) with one row and one byte column (ASDF_METADATA) holding a complete ASDF file of 60–72 KB, with its own blocks and block index. `infra/fits/asdf_hdu.rs` finds it by its header (EXTNAME='ASDF', one <n>B column, one row), reads the cell as raw bytes from the data area without going through the table reader, checks the #ASDF magic and hands it to the ASDF parser, so the gWCS and `meta.wcsinfo` of the exposure are available beside the FITS header.

9.6 Ramps (v0.6.6)

A 4-D FITS image with `NAXIS4 > 1` and `NGROUPS/NINTS` cards, or a `DATAMODL` of `Level1bModel` or `RampModel`, opens as a ramp instead of a cube of `NINTS × NGROUPS` planes; the primary header supplies the ramp cards and the SCI header the WCS and units. Roman L1 ASDF resultant stacks open the same way.

10 WebGPU Compute Pipeline

WGSL fragment shader implementing MTF with symmetric denominator guard. `r32float` texture, 6-vertex fullscreen quad. Uniform buffer (32 bytes): `dataMin`, `dataMax`, `shadow`, `midtone`, `highlight`, `W`, `H` and a padding word. CPU worker fallback with identical `stfTransfer` function.

10.1 Display Path Unification (v0.6.0, extended in v0.6.3 and v0.6.5)

The shader evaluates the six scale modes (`mtf`, `linear`, `log`, `sqrt`, `asinh`, `power`) and samples a $256 × 1$ LUT texture for the colormap, so a colormap change uploads 1 KiB rather than re-rendering on the CPU. Limits come from the four limit modes (`minmax`, `zscale`, `percentile`, `user`) computed in Rust by `compute_scale_limits_cmd`; the LUT itself comes from `get_colormap_lut_cmd`.

The point of the exercise is that the stretch, the limit algorithms and the float-to-byte rule are written once, in `core/imaging/scale.rs`, and three consumers agree on them: the WGSL shader, the TypeScript worker fallback, and the headless server which returns a PNG. The parity is asserted rather than assumed — a vtest case sweeps every scale kind and checks the WGSL port against the canonical transfer within `f32` rounding.

Since 0.6.3 there are 13 colormaps: nine sequential (gray, viridis, inferno, magma, plasma, cividis, heat, cool, rainbow) and four diverging (RdBu, coolwarm, bwr, seismic, from the matplotlib knots). Symmetric limits mirror the resolved pair about a centre c (default 0): $(lo, hi) \mapsto (c - a, c + a)$ with $a = \max(|lo - c|, |hi - c|)$, falling back to a half-width of $\max(1, |c| 2^{-20})$ for a degenerate pair. No-data pixels (NaN, infinity, exact 0) take a per-colormap colour that ignores `invert`: the first LUT entry for the sequential maps and neutral grey #404040 for the diverging ones, identically on the desktop shader, the worker and the server. Since 0.6.5.0 a colorbar under the mono GPU viewer shows the current colormap with value ticks placed through the current stretch and limits, the BUNIT unit and the no-data swatch.

Device loss is recoverable. A failed initialisation used to be cached for the session, so one transient adapter error left the app on the CPU path until restart; the retry path now re-probes. When WebGPU is unavailable, the CPU viewer shows the auto-stretched preview with zoom, pan, regions, the grid and the compass, and disables the stretch, limit and colormap controls with the reason.

GPU scope

The GPU is used for display only. Stacking, alignment, drizzle, deconvolution, background extraction, PixelMath, wavelets, line fits, reprojection and the compose pipeline all run on the CPU in Rust, parallelised with `rayon`. No processing result depends on a GPU being present.

11 Headless HTTP Server

AstroBurst ships a second binary that exposes the same imaging core over HTTP, with no window, no WebView and no Tauri runtime. It is an `axum` 0.7 service on `tokio`, built around short-lived sessions that hold a per-session image cache, so a script or notebook can open a FITS or ASDF file once and then issue many inspection calls against the decoded array instead of re-reading the file each time. The headless server was contributed by Jae-Joon Lee (<https://github.com/leejjoon>); the Cargo manifest records this on the server dependency block and on the `[[bin]]` entry, and every source file under `src-tauri/src/server/` carries the same note.

The server is gated behind the `server` Cargo feature, which is *not* in default. The feature pulls in `axum`, `dashmap`, `uuid`, `tokio-stream`, `tokio-util` and `env_logger`, all declared optional; `tower` is a development dependency of the server tests only. The binary is `astroburst-server` (`src/server/main.rs`) and declares `required-features = ["server"]`, so it is only built when that feature is on. Because the desktop feature `tauri` is a default, the server build turns the defaults off and re-enables the non-UI ones explicitly:

Build

```
cargo build --no-default-features \
  --features server,astrometry-net,asdf-full,vizier
```

On start the process reads its configuration from the environment, initialises `env_logger`, resolves the catalog cache directory and the Vizier endpoint (`ASTROBURST_CATALOG_CACHE_DIR`, `ASTROBURST_VIZIER_URL`, as the desktop app does), spawns the TTL cleaner and serves on the

configured address. Every response carries an `x-request-id` header, echoed from the request when present and otherwise a fresh UUIDv4, applied by an `axum middleware::from_fn` layer over the whole router.

Variable	Meaning	Default
<code>ASTROBURST_BIND</code>	Listen address	<code>127.0.0.1:8080</code>
<code>ASTROBURST_SESSION_TTL</code>	Idle seconds before a session is evicted	<code>900</code>
<code>ASTROBURST_SESSION_MAX</code>	Concurrent sessions	<code>8</code>
<code>ASTROBURST_JOBS_MAX</code>	Concurrent long-running jobs	<code>4</code>
<code>ASTROBURST_CACHE_MAX_ENTRIES</code>	Per-session cache entries	<code>32</code>
<code>ASTROBURST_CACHE_MAX_BYTES</code>	Per-session cache bytes	<code>2 GiB</code>
<code>ASTROBURST_CLEANUP_INTERVAL</code>	Seconds between TTL sweeps	<code>60</code>
<code>ASTROBURST_LOG_LEVEL</code>	Default <code>RUST_LOG</code> value	<code>info</code>

Table 8: Server configuration (`server/config.rs`). An unparseable value is reported on stderr and replaced by the default rather than aborting startup.

11.1 Session Model

A session is created by `POST /v2/sessions` (or the identical `POST /sessions`), which answers 201 with a UUIDv4 `session_id`. Sessions live in a `dashmap` keyed by that id; there is no authentication and no per-session credential.

Each session owns four things (`server/session.rs`):

- an `ImageCache` — the same LRU cache the desktop app uses — bounded by both `cache_max_entries` and `cache_max_bytes`;
- a map of jobs, keyed by job id;
- the v2 state: a monotonic counter, an `active_ref` under an `RwLock`, and a map of `ImageMeta` records;
- a `last_accessed` timestamp.

Images are referenced by name inside the session rather than by path. The v2 routes mint a ref on each producing call — `img_0`, `img_1`, `cutout_0`, `bin_0` and so on, from a per-session counter — unless the caller supplies `name`. Each ref carries an `ImageMeta` record holding `image_ref`, `source`, `hdu`, `array`, `plane_ref`, `is_dq`, `width`, `height`, `wcs_present` and `extname`. The most recently opened, cut or switched image becomes the session’s `active_ref`, and every consuming endpoint treats a missing `ref` field as “use the active ref”, failing with 400 if there is none. The older slot-based routes instead take a caller-chosen `slot` string, defaulting to the file path.

Lifetime is bounded on three axes. Session creation fails with 503 once `session_max` sessions exist. Any request that resolves a session touches its timestamp through the `SessionExtractor`, and `POST /v2/sessions/:sid/keepalive` exists to do only that. A background task ticks every `cleanup_interval`, skips any session with a running job, and removes sessions idle longer than `session_ttl`, logging the eviction. `DELETE /v2/sessions/:sid` removes one immediately.

Eviction inside a session is independent of session lifetime: the image cache drops least-recently-used entries when either bound is exceeded, which can silently invalidate a ref the client still holds. The server reconciles rather than pretending otherwise — `prune_evicted_meta` drops metadata whose cache entry is gone, and `reconcile_active_ref` additionally clears `active_ref` when it points at an evicted image. `GET /v2/sessions/:sid` and `GET /v2/sessions/:sid/images` run that reconciliation before reporting, so a listing never advertises an image the cache no longer holds.

11.2 The v2 API Surface

Method	Path	Purpose
<i>Liveness</i>		
GET	/health	Status, version, active and cumulative session counts
<i>Session lifecycle</i>		
POST	/v2/sessions	Create a session, return its id
GET	/v2/sessions/:sid	Active ref, image count, cache bytes
DELETE	/v2/sessions/:sid	Drop the session and its cache
POST	/v2/sessions/:sid/keepalive	Refresh the idle timer
<i>Images and structure</i>		
POST	/v2/sessions/:sid/open	Open a FITS or ASDF plane, become the active ref
POST	/v2/sessions/:sid/hdu	Re-open the active image at another HDU or array
GET	/v2/sessions/:sid/images	List refs with their metadata
GET	/v2/sessions/:sid/structure	HDU listing of the source file (FITS only)
GET	/v2/sessions/:sid/header	Header cards, optionally glob-filtered by keys
GET	/v2/sessions/:sid/wcs	WCS summary: projection, CRPIX/CRVAL, CD, scale, rotation, parity, SIP and its fit residuals, WCS kind and gWCS keys
<i>Astrometry</i>		
POST	/v2/sessions/:sid/wcs/pix2sky	Batch pixel $\rightarrow$ sky, in a named frame
POST	/v2/sessions/:sid/wcs/sky2pix	Batch sky $\rightarrow$ pixel
POST	/v2/sessions/:sid/wcs/separation	Angular separation between two sky or two pixel points
POST	/v2/sessions/:sid/wcs/grid	Coordinate grid lines and labels at a given density
<i>Measurement</i>		
POST	/v2/sessions/:sid/pixel	Probe a pixel with an odd $n \times n$ box, plus DQ, ERR and WAVELENGTH companions
POST	/v2/sessions/:sid/stats	Region statistics: exact moments, sigma clipping, percentiles, noise
POST	/v2/sessions/:sid/histogram	Binned histogram over a region, auto or explicit range
<i>Derived images and display</i>		
POST	/v2/sessions/:sid/cutout	Cut a region into a new ref, optionally shifting the WCS
POST	/v2/sessions/:sid/bin	Block-mean downsample into a new ref
POST	/v2/sessions/:sid/render	Render a region to PNG with limits, stretch, colormap, overlays

Table 9: Every route registered in `server/router.rs` under `/v2`, plus `/health`. Paths are literal; `:sid` is the session id.

The v2 endpoints share one region grammar (`server/v2/region.rs`): a request may pass a pixel rectangle, a sky rectangle in arcminutes around an RA/Dec, or a full shape — the core `RegionShape` set — in pixel or sky coordinates. A region that leaves the frame is rejected with `region_out_of_bounds` unless the caller passes `clip: true`, and a sky region on an image without WCS is rejected with `wcs_required`.

`POST /v2/sessions/:sid/render` is the only route that does not answer JSON: it returns `image/png` and puts the resolved parameters — the region actually used, `vmin/vmax`, the limit

algorithm, stretch, colormap, the binning factor applied to honour `max_dim`, the PNG's arcsec-per-pixel scale and the fraction of pixels clipped at each end — into an `x-render-resolved` JSON header. Crosshair and scalebar overlays are drawn into the buffer before encoding. The `scale` block accepts the 13 colormaps and `symmetric/centre` (alias `center`), and the resolved header echoes them with any notes.

WCS on the server (0.6.7). Every server route that needs a WCS goes through the same loader as the desktop app (Section 4), keyed by the source file of the image ref, so a JWST _cal or Roman image opened in a session is served by its gWCS and a derived ref (a cutout or a binned image) by its own header. The `wcs` route returns the keys of `get_wcs_info`: `sip_max_err_px`, `sip_inv_err_px`, `wcs_kind`, `gwcs_steps`, `gwcs_frames`, `gwcs_source`, `gwcs_vs_header_sip_max_mas/_px` and `gwcs_refusal`. `wcs/pix2sky` applies the gWCS bounding box and returns null coordinates for a pixel outside it. The stricter header rule (a celestial CTYPE pair) applies here too.

Alongside these, the router keeps an earlier slot-based surface: `POST /sessions` and, under `/sessions/:sid`, `fits/open` and `fits/header` for I/O; `image/render`, `image/stf` and `image/viewport` for STF-based PNG output; `stacking/stack`, `stacking/drizzle` and `pipeline/run` for batch work; and the three job routes below. These address images by slot and do not participate in the v2 ref or active-ref model.

11.3 Job Model

Only three routes start work that can outlive a request: `stacking/stack`, `stacking/drizzle` and `pipeline/run`. All three follow the same contract (`server/job.rs`, `server/handlers/`):

- **Admission.** The handler validates its parameters, then takes a permit from a Semaphore sized at `jobs_max` with `try_acquire_owned`. No permit means an immediate 429; the queue never builds up. Validation includes refusing one-shot-colour lights (BAYERPAT or COLORTYP) for stacking with alignment, drizzle and the aligned pipeline (0.6.5.0), and, for `pipeline/run`, the frame-card checks of Section 6 (0.6.7): a refused calibration set is a 400 before any job starts, and the warnings are returned in the 202 body. The pipeline body accepts `flat_dark_paths` since 0.6.7.
- **Submission.** A Job is created with a bounded 256-slot event channel, registered in `session.jobs`, and the work is moved onto `spawn_blocking` holding the permit. The handler answers 202 Accepted with `job_id`, `status`: "running" and the cache slot(s) the result will be stored under. The pipeline route additionally returns any DQ cosmetic warnings raised while inspecting the input frames.
- **Polling.** `GET /sessions/:sid/jobs/:jid` returns `id`, `action`, `status` (`running`, `done`, `error`, `cancelled`), `pct`, and `started_at/completed_at` as milliseconds since the epoch.
- **Streaming.** `GET /sessions/:sid/jobs/:jid/stream` is a Server-Sent Events stream of `progress`, `complete` and `error` events, with a 30-second keep-alive. The receiver is taken out of the job on first subscribe, so a second subscriber gets 409.
- **Cancellation.** `DELETE /sessions/:sid/jobs/:jid` cancels a running job's `CancellationToken`, marks it cancelled and returns the job record. Transitions out of `running` go through a compare-and-exchange, and `set_cancelled` is a plain store, so a worker that finishes after the cancel cannot overwrite the cancelled state with `done` or `error`.

Results are written into the session's image cache under the requested slot names; there is no separate result store and no download route for them. Sessions with a running job are skipped by the TTL cleaner, so a long stack cannot have its own session evicted underneath it.

11.4 What Is Shared, and What Is Not

The server is a second front end over the library crate, not a reimplementation. It links `astroburst_lib` and calls the same core modules the Tauri commands call:

- `core/imaging/scale.rs` — `LimitMode`, `StretchKind`, `resolve_limits`, `normalize_and_stretch`: used by `server/v2/render/handler.rs` and by `cmd/visualization/mod.rs`, so limits and stretches resolve identically on both;
- `core/imaging/region.rs` — the `shape` set, `shape_to_pixel`, `masked` values: used by `server/v2/region.rs` and by `cmd/regions.rs`, `cmd/statistics.rs`, `cmd/cutout.rs`;
- `core/imaging/cutout.rs` — `resolve_cutout_rect`, `cut_plane_within`, `shift_header`, `reported_ltv`: used by `server/v2/cutout.rs` and `cmd/cutout.rs`;
- `core/imaging/pixel_probe.rs` — `probe_pixel`, `probe_companions`, `data_unit`: used by `server/v2/pixel.rs` and by `cmd/statistics.rs` and `cmd/visualization/mod.rs`.

The same holds for `core/imaging/colormap.rs`, `core/imaging/statistics.rs`, `core/astrometry/` (`wcs.rs`, the `gwcs/` evaluator, `grid.rs`, `frames.rs`), `core/stacking/` (the rejection kernel, frame cards, consistency checks and masters) and the infrastructure layer — `infra/cache.rs`, `infra/image_source.rs` and `infra/wcs_source.rs` — all of which both binaries import.

The split is in the command layer, not the core. Radial profiles, line cuts, exclude regions and DS9 `.reg` import/export exist as core functions (`radial_profile` and `line_cut` in `core/imaging/region.rs`, and `core/imaging/region_file.rs`) but are reachable only through `cmd/regions.rs`; no route in `server/router.rs` exposes them. Likewise, the `exclude_dq` flag that drives `resolve_dq_mask` appears only under `cmd/` — in `cmd/analysis/mod.rs`, `cmd/regions.rs` and `cmd/statistics.rs` — and is absent from the server's `StatsParams` and `HistogramParams`.

DQ on the server

The asymmetry is narrower than “no DQ”. `POST /v2/sessions/:sid/pixel` does load the DQ and ERR companions of the probed plane and returns decoded DQ flags for the sampled pixel. What the server lacks is *DQ-masked* aggregation: its stats and histogram endpoints have no exclusion flag, so they measure every finite pixel in the region regardless of its DQ state.

Further desktop-only surface: photometry and the gain model, catalogs, cube science (spectra, moments, line fits, PV), ramps, contours, time series, observation geometry, the processing tools and the compose wizard have no server routes, and `histogram` rejects `render_png: true` with a `not_implemented` code rather than plotting server-side.

11.5 Error Model

All failures go through one enum in `server/error.rs` and serialise to a single envelope, `{"success": false, "error": {"code", "message", "hint"?}}`, where `hint` is present only on the variant that carries one.

Status	Code	Raised for
400	<code>bad_request</code>	Malformed path or body, unknown colormap, stretch, algorithm or frame, both <code>hdu</code> and <code>array</code> given, zero or oversized bin factor, no active image in the session, one-shot-colour lights for an aligned stack or drizzle, a calibration set refused by the frame-card checks
400	<code>pixel_out_of_bounds</code>	Pixel probe outside the array; hint gives the valid extent
400	<code>region_out_of_bounds</code>	Region off-image, non-overlapping, clipped without <code>clip: true</code> , or a cutout exceeding the session memory budget
400	<code>wcs_required</code>	Sky coordinates requested on an image with no header, no usable WCS, or a degenerate pixel scale
400	<code>not_implemented</code>	<code>render_png</code> on the histogram endpoint
404	<code>not_found</code>	Unknown session, image ref, slot or job id
409	<code>conflict</code>	Second SSE subscriber on one job
429	<code>too_many_requests</code>	No job permit available
503	<code>service_unavailable</code>	Session cap reached
500	<code>internal_error</code>	Any anyhow error reaching the boundary, including a panicking blocking task

Table 10: Status codes produced by `AppError`. The four typed 400 codes let a client distinguish a bad request from an out-of-frame one without parsing prose.

Two details are worth noting for clients. The 429 message, "`job queue full (max N concurrent)`", names the configured `jobs_max`; the status code, not the text, is the contract. And errors that originate in the core — `RegionError`, `CutoutError`, `ProbeError` — are mapped explicitly to 400 with a hint, so a caller mistake is never reported as 500. The exception is plane loading, where only the two recognised messages (no ASDF arrays, no HDU index) become 400 and everything else is surfaced as `internal_error`.

12 Testing and Continuous Integration

CI job	Runner	Command
<code>typecheck</code>	Ubuntu	<code>npx tsc --noEmit</code> , then <code>npx vite build</code>
<code>lint</code>	Ubuntu	<code>npx eslint src --max-warnings 0</code>
<code>backend-tests</code>	Windows	<code>cargo test --lib</code>
<code>backend-tests-unix</code>	Ubuntu, macOS	<code>cargo test --all-features</code> (library and server binary)
<code>frontend-tests</code>	Ubuntu	<code>pnpm test (vitest)</code>

Table 11: The five jobs of `.github/workflows/ci.yml`, run on every pull request and on pushes to main.

Suite sizes at release. `cargo test --lib`: 2389 passed, 0 failed, 29 ignored. `cargo test --all-features --bin astroburst-server`: 190 passed. `npx vitest run`: 2962 tests in 199 files. `cargo check --all-features`, `tsc` and `eslint` report no warning. At the start of the 0.6.7 round the same suites counted 2073, 185 and 2637 tests in 175 files.

Ignored tests are real-data tests. The 29 `#[ignore]` tests read local data and skip, with a message, when it is absent: the gWCS truth and data sets (`ASTROBURST_GWCS_TRUTH_DIR`, `ASTROBURST_GWCS_DATA_DIR`), MAST downloads (`ASTROBURST_MAST_DIR`), NIRSpec and MIRI MRS products (`ASTROBURST_DOCUMENTS_FITS_DIR`, `ASTROBURST_NIRSPEC_DIR`, `ASTROBURST_MIRI_MRS_DIR`), ramp pairs (`ASTROBURST_RAMP_PAIR_DIR`), the git-ignored `exampleFits/sample-data/heavyTest`

mosaics, and timing checks, among them the release-mode gWCS bench. They run with `cargo test --lib -- --ignored`.

Fixtures. The gWCS evaluator is tested against ASDF fixtures committed under `src-tauri/tests/fixtures/gwcs/` (one per transform family and per JWST instrument chain, written with gwcs 1.0.3 and asdf-astropy, BSD-3 provenance recorded in `PROVENANCE.md` and the gwcs licence beside them) and seven trimmed truth files with forward and inverse points computed by gwcs. A vitest parses the Rust filter table and requires it to match the TypeScript table entry for entry.

Practice. Every change of the 0.6.7 round was written test-first: a typed stub that compiles, a failing assertion recorded, then the implementation. The round’s UI-visible behaviour was also accepted in the running application, driven over the WebView2 remote-debugging protocol by scripts kept outside the repository: the 54 cases of the round passed, together with the 331 checks of the line-fit regression driver (3 documented skips). The four final items (dock onboarding, gWCS arithmetic operators, Pipeline frame exclusion and the SPCC label) were verified by their tests and the final gate run; their in-app cases were not part of that acceptance run.

13 Performance Benchmarks

There is no benchmark suite in the repository. The first table is historical; the second lists the figures measured during the 0.6.6 and 0.6.7 rounds, each a single run with its source named.

Operation	Input Size	Time	Notes
Single FITS processing	4096×4096	120 ms	533 MB/s
Batch (10 frames)	10×64 MB	450 ms	1.4 GB/s (Rayon)
Histogram + auto-STF	4096×4096	35 ms	NaN-safe
Star detection ($\sigma=5$)	4096×4096	80 ms	~3000 stars
Sigma-clip stack	10×64 MB	2.1 s	5 iters
Drizzle 2×	10×64 MB	4.2 s	Flat accumulator
Channel blend (3ch)	4096×4096 × 3	0.4 s	+ auto STF + lum FITS
Affine alignment	4096×4096	0.8 s	80 stars, RANSAC
Masked stretch (10 iter)	4096×4096	1.2 s	Star mask + MTF
WebGPU STF render	4096×4096	8 ms	GPU compute
WB (non-destructive)	4096×4096 × 3	45 ms	ORIG read + factor

Table 12: Historical benchmarks (AMD Ryzen 9 7950X, 64 GB DDR5, NVMe), measured on the v0.4.6 build and not re-measured since; treat them as orders of magnitude. Several kernels have changed (rejection, masked stretch, alignment).

Operation	Time	Conditions and source
gWCS forward evaluation	271 ns/pt	NIRCam chain, 10^5 random points in the bounding box, release build, <code>gwcs_bench</code>
gWCS analytic inverse	274 ns/pt	same
gWCS exact inverse (Newton)	$3.75 \mu\text{s}/\text{pt}$	same; 2.0 iterations on average, maximum error 1.1×10^{-8} px
Align by WCS, F444W onto F200W	1.95 s	14344×8589 output (123 Mpx), reprojection plus point-source residual, in the app (optimised development build)
Line-fit maps, NIRSpec G395H cube	64 ms	53×55 spaxels, 684 fitted, Br α window of 18 channels, README screenshot run (0.6.6)
Exact statistics, NIRCam F444W	805 ms	5668×4159 , full frame, README screenshot run (0.6.6)
Star detection, NIRCam F444W mosaic	631 ms	82 663 sources, brightest 200 kept, README screenshot run (0.6.6)
Photometry table, 200 stars	195 ms	same mosaic, README screenshot run (0.6.6)

Table 13: Figures measured on the development machine during the 0.6.6 and 0.6.7 rounds.

14 Dependencies

The default feature set is `tauri`, `astrometry-net`, `asdf-full` and `vizier`; the headless server is built instead with `--no-default-features --features server,astrometry-net,asdf-full,vizier`. The 0.6.7 round added no dependency; it enabled the `float_roundtrip` feature of `serde_json`.

15 Known Limitations

The first five items are the competitive gaps against Siril and PixInsight that the roadmap plans to close (Section 16, N1–N5).

1. **Plate solving is online only (N1).** Solving uploads the frame to `nova.astrometry.net` and needs an API key; there is no local index or solver. Since SPCC and every WCS feature need a solution, most amateur frames depend on it.
2. **No RAW, TIFF or XISF input (N2).** Camera RAW files, TIFF (including Astro-TIFF) and XISF are not read; FITS, ASDF and ZIP containers are.
3. **No masks, undo, history or presets (N3).** Masks are not first-class objects, and beside the fixed-stage processing chain and the composite chain there is no undo, action history, process container or saved preset.
4. **Registration of sequences is basic (N4).** The batch Pipeline registers each frame against the first by phase correlation or the affine matcher and now excludes what it cannot register; there is no local normalization, mosaic assembly, comet alignment or distortion-aware sequence registration, and integration holds every frame in memory.
5. **SPCC is a blackbody approximation (N5).** Expected colours come from a Planck spectrum at each channel’s effective wavelength, driven by Gaia BP–RP; Gaia XP spectra are not integrated through filter and sensor curves.
6. **gWCS coverage.** Imaging gWCS only: spectroscopic, slit and IFU gWCS are not evaluated and those files keep their header WCS; Roman `_uncal` files do not look up their `_wcs.asdf` sidecar; transforms outside the supported set (`tabular`, `fix_inputs`, `jwst_pipeline/*`) fall back to the header with the reason shown. A derived Roman FITS carries the fitted TAN-SIP, accurate to the stated residual, not the gWCS.

Crate	Version	Purpose
tokio	1	Async runtime; every command is moved off the UI thread with <code>spawn_blocking</code>
serde	1	Derived serialization for the IPC and server payloads
serde_json	1	JSON command responses; <code>float_roundtrip</code> keeps every digit of numbers sent by the frontend
serde_yaml	0.9	ASDF YAML tree parsing (<code>infra/asdf/parser.rs</code> , <code>tree.rs</code>), gWCS trees included
anyhow	1.0	Error propagation inside commands
thiserror	2.0	Typed errors for the region, cutout, pixel-probe and gWCS cores
ndarray	0.16	N-dimensional arrays, with the <code>rayon</code> and <code>serde</code> features
rayon	1.10	Data-parallel batch processing
rand	0.8	Seeded RNG for the synthetic data generator (<code>core/synth</code>) and tests
image	0.25	PNG encoding, 8 and 16 bit (<code>cmd/helpers.rs</code>)
rustfft	6.2	FFT for deconvolution, phase correlation and the power spectrum
memmap2	0.9	Memory-mapped FITS and ASDF reads, lazy cube frames
flate2	1.0	zlib for ASDF blocks and GZIP_1/GZIP_2 FITS tiles
wcs	0.4	wcs-rs projection engine (~20 FITS projections); SIP is applied by AstroBurst's own math
zip	2	Transparent .zip FITS containers (<code>infra/fits/dispatcher.rs</code>)
tempfile	3.8	Scratch files for the plate-solve upload and for tests
dirs	5.0	Platform data, config and cache directories (the catalog cache)
regex	1.10	Header keyword discovery (<code>core/metadata/header_discovery.rs</code>)
base64	0.22.1	Base64 previews in the batch pipeline response
log	0.4.29	Logging facade used by the commands and the server
libc	0.2	Linux target only: <code>fstatfs</code> identifies a network filesystem before a file is memory-mapped
tauri-build	2.5	Build dependency: Tauri context and asset generation
tower	0.5	Development dependency: <code>ServiceExt</code> in the server test harness

Table 14: Unconditional, build and development dependencies, from `src-tauri/Cargo.toml` at v0.6.7. Complex arithmetic is in `math/complex.rs`; `num-complex` and `tower-http` are no longer declared.

Crate	Version	Purpose (gating feature)
tauri	2.10	Application framework, IPC, asset protocol — <code>tauri</code>
tauri-plugin-dialog	2.6.0	Native file dialogs — <code>tauri</code>
tauri-plugin-fs	2.2.0	Filesystem access from the webview — <code>tauri</code>
tauri-plugin-shell	2.3	External process and shell access — <code>tauri</code>
tauri-plugin-opener	2.3	Open files and URLs in the system handler — <code>tauri</code>
axum	0.7	HTTP router and request-id middleware of the headless server — <code>server</code>
dashmap	6	Concurrent session and image maps — <code>server</code>
uuid	1	v4 session, job and request identifiers — <code>server</code>
tokio-stream	0.1	SSE job-event stream — <code>server</code>
tokio-util	0.7	Cancellation tokens for jobs — <code>server</code>
env_logger	0.11	Logger initialisation for the server binary — <code>server</code>
request	0.12	Astrometry.net upload and Vizier, <code>rustls-tls</code> and <code>multipart</code> — <code>astrometry-net</code> ; <code>vizier</code> adds <code>request/blocking</code>
bzip2	0.5	bzip2 ASDF blocks — <code>asdf-full</code>
lz4_flex	0.11	lz4 ASDF blocks — <code>asdf-full</code>

Table 15: Optional dependencies and the Cargo feature that gates each.

7. **Tabulated axes and tables.** Only separable one-axis `-TAB` tables are read; variable-length P/Q columns are not; a PV diagram of a `-TAB` cube is refused when the table departs from linear by more than 1 % of its step.
8. **No distortion-correcting export.** Compose Align reprojects channels through their full WCS (SIP or gWCS), but there is no tool that resamples a single image onto an undistorted grid.
9. **Provenance gaps.** The RGB drizzle FITS does not carry the stack cards (NCOMBINE, ABCOMB, ABRZSCL); Geometry writes the science plane only.
10. **ASDF is read-only.** The reader is complete (memory-mapped, lazy blocks, gWCS); no writer is implemented, so Roman products are exported as FITS.
11. **No MAST API integration.** Observatory data must be downloaded before it is opened.
12. **No session persistence.** The dock layout, display settings, per-file regions and panel preferences survive a restart through `localStorage`; the loaded files, the processing chains and the composite do not. There is no project file.
13. **WebGPU is preferred, not required.** The display path needs a WebGPU-capable WebView; otherwise the CPU viewer is used without the stretch, limit and colormap controls.
14. **Server parity is partial.** Rendering, region statistics, histograms, cutouts, WCS queries, stacking, drizzle and the pipeline are available over HTTP. Radial profiles, line cuts, exclude regions, `.reg` import/export, DQ-masked statistics, photometry, catalogs, cube science, ramps, the processing tools and the compose wizard are desktop-only, and the Python client covers the v1 routes only.

16 Roadmap

Feature	Priority	Status
Batch calibration and stacking pipeline	Medium	Done (v0.3.0)
Star-based affine alignment	High	Done (v0.4.0)
ComposeWizard pipeline	High	Done (v0.4.2)
Masked stretch	High	Done (v0.4.2)
SPCC calibration (blackbody)	High	Done (v0.4.2)
Non-destructive WB/SCNR	High	Done (v0.4.5)
STF GPU/CPU consistency	High	Done (v0.4.5)
Tone curves (AdjustStep)	High	Done (v0.4.6)
Cache-only processing	High	Done (v0.4.6)
Auto-STF preview	Medium	Done (v0.4.6)
Headless HTTP server and Python client	Medium	Done (v0.5.5.2)
Star removal (starless plus stars layer)	Medium	Done (v0.5.6.1)
Display controls: scale modes, limit modes, colormaps	High	Done (v0.5.8.1)
Multi-extension FITS: any HDU, any ASDF array, ERR/DQ	High	Done (v0.5.8.1)
DQ flag decoding, tables and overlays	High	Done (v0.5.8.1)
Interactive regions with DS9 .reg exchange	High	Done (v0.5.8.2)
Pixel rejection family and frame normalization	High	Done (v0.5.8.3)
DBE spline background, PixelMath, LHE and HDRMT	High	Done (v0.5.8.3)
Exact statistics with k-sigma noise evaluation	High	Done (v0.5.8.3)
Photometric calibration, spectral axes, moment maps, cutouts, WCS grid, Gaia panel	Medium	Done (v0.5.8.4)
GPU viewer follows the processed result	High	Done (v0.6.0)
Measurement round: batch photometry, calibrated regions, line measurement, contours, time series	High	Done (v0.6.2)
Line list, spectrum comparison, observation geometry, PV diagrams, measurement log	Medium	Done (v0.6.3)
Processing on the colour composite	Medium	Done (v0.6.4)
Dockable tool windows, exclude regions, x1d comparison, Write WCS	High	Done (v0.6.5.0)
IFU line-fit maps; JWST ramps with quick slope	Medium	Done (v0.6.6)
gWCS evaluation for JWST _cal and Roman L2; TAN-SIP fit	High	Done (v0.6.7.0)
Calibration integrity: frame-card checks, flat-darks, temperature groups	High	Done (v0.6.7.0)
Compose Align through the WCS; level matching by median and MAD	High	Done (v0.6.7.0)
Curves on files, de-banding on files, rotate and flip	Medium	Done (v0.6.7.0)
Flatpak	Medium	Community package (FlatPark)
N1: offline plate solving (hinted, then blind)	High	Planned
N2: RAW, TIFF and XISF read	High	Planned
N3: masks, undo, history and presets	High	Planned
N4: sequence registration, local normalization, mosaics, comets	Medium	Planned
N5: SPCC on Gaia XP spectra through filter curves	Medium	Planned
MAST query and download	Medium	Planned

Full GPU compute pipeline	Low	Planned (v1.0)
---------------------------	-----	----------------

Table 16: Done rows are dated from the release that first contains them; four-component versions are point releases grouped in `CHANGELOG.md` under the nearest published entry. The planned rows N1–N5 are the five gaps of Section 15, numbered as in the plan of 2026-10-10; that plan schedules N5 before N4 and first extracts shared foundations (a HEALPix catalogue reader, a triangle matcher, a polynomial fitter, a catalogue store with one consent flow, a camera database) so the gaps do not each build their own. The Flatpak is maintained by a community packager, not by the project, and installs the official `.deb` unmodified.

A Sample Test Data

HST/WFPC2 narrowband FITS in `exampleFits/sample-data/`: `502nmos.fits` ([OIII] 502 nm, Blue), `656nmos.fits` (H α 656 nm, Green), `673nmos.fits` ([SII] 673 nm, Red). 1600 $\times$ 1600 float32, WCS TAN. Public domain (NASA/ESA). The same directory carries `synthetic_catalog.csv`, the ground-truth source list emitted by the synthetic data generator, which is what photometry and alignment are validated against.

The real-data tests and the release acceptance used, in addition, the JWST NIRCam mosaics of the Pillars of Creation (`jw02739`, kept locally in the git-ignored `exampleFits/sample-data/heavyTest/`), JWST NIRCam, MIRI and FGS `_cal` files, the tweakreg-corrected `_crf` products of program 1475, NIRSpec IFU cubes, `_uncal`, `_rate` and `_cal` frames of program 1266, MIRI MRS cubes of program 2016 and a Roman WFI01 F129 L2 `_cal.asdf`. None of these are distributed with the repository.